



UNIVERSITY OF
CAMBRIDGE

Department of Engineering

Towards better Neural Samplers for Boltzmann Distribution

Author Name: Luran Wang

Supervisor: José Miguel Hernández Lobato

Date: 2 June 2025

I hereby declare that, except where specifically indicated, the work submitted herein is my own original work.

Signed Luran Wang date 02. 06. 2025

Abstract

Efficiently sampling from complex, high-dimensional probability distributions, particularly the Boltzmann distribution prevalent in statistical mechanics and molecular modeling, is a critical challenge. This thesis focuses on developing and evaluating **better neural samplers for Boltzmann distribution**, aiming to enhance the accuracy and computational efficiency of generating representative samples from these physically significant target densities. We first explore the development of a **Multi-Step Diffusive KL (DiKL) Sampler**. This approach utilizes a training objective based on diffusive Kullback-Leibler divergence, demonstrating that iterative refinement through multiple steps can improve sample quality when targeting Boltzmann distributions. Building on the principles of physics-informed machine learning and the Stochastic Differential Equation (SDE) formulation of Langevin dynamics, we then introduce a novel **Langevin Sampler with Learned Dynamics**. This method incorporates learned, position-dependent components, such as an adaptive step size, directly into the Langevin SDE governing sampling from a Boltzmann distribution. We analyze its bias and demonstrate through multi-step and auto-regressive applications that this approach significantly enhances sample quality and efficiency. Finally, to ensure rigorous convergence to the target Boltzmann distribution while leveraging learned efficiencies, we develop a **Metropolis-Adjusted Langevin Sampler with Learned Dynamics**. This method combines the adaptive proposals from the learned Langevin sampler with a Metropolis-Hastings acceptance criterion. Experimental results on challenging benchmark systems confirm the superior performance of these advanced neural samplers, particularly the Langevin-based methods with learned dynamics, in terms of both sample accuracy and computational speed when sampling from Boltzmann distributions. In summary, this thesis contributes novel and effective neural samplers, offering more robust and practical tools for complex scientific applications reliant on accurate Boltzmann sampling.

Contents

1	Introduction	3
1.1	Contributions	3
1.2	Outline	4
2	Background	5
2.1	The Boltzmann Distribution and its Central Role in Molecular Modeling	5
2.1.1	Application: Molecular Modeling	6
2.2	Sampling Methods	7
2.2.1	Langevin Sampling	7
2.2.2	Metropolis-Adjusted Langevin Algorithm (MALA)	9
2.3	Neural Samplers	10
2.3.1	Neural Samplers with Diffusive Reverse KL Divergence	11
3	Multi-Step Diffusive KL Divergence Sampler	14
	Introduction	14
3.1	Methods	14
3.2	Experiments and Results	15
3.2.1	Experiment Setting	15
3.2.2	Analysis	17
3.3	Summary	21
4	Langevin Sampler with Learned Dynamics	22
	Introduction	22
4.1	Methods for Learned Langevin Dynamics	23
4.1.1	Multi-Step Langevin Sampling for Enhanced Generative Capacity	24
4.1.2	Auto-Regressive Langevin Sampling for Sequential Refinement	25
4.1.3	Bias Analysis of Position-Dependent Step Sizes	26

4.1.4	Langevin Samplers as Post-Processors for Enhanced Sampling in Complex Systems	28
4.2	Experiments and Results	29
4.2.1	Experiment Setting	29
4.2.2	Analysis	30
4.3	Summary	34
5	Metropolis-Adjusted Langevin Sampler with Learned Dynamics	35
	Introduction	35
5.1	Method for Learning Adaptive MALA	36
5.1.1	Multi-Step and Auto-Regressive MALA Samplers	38
5.2	Experiments and Results	39
5.2.1	Experiment Setting	39
5.2.2	Analysis of Learned MALA Performance	39
5.3	Summary	44
6	Conclusion	45
6.1	Limitations	45
6.2	Future Works	46
A	Risk Assessment	50

Chapter 1

Introduction

The accurate and efficient generation of samples from complex, high-dimensional probability distributions is a fundamental challenge across numerous scientific and engineering disciplines [23]. Of particular importance is the **Boltzmann distribution**, describing the equilibrium state of systems. Fields such as molecular modeling, for instance, heavily rely on robust techniques to sample from Boltzmann distributions to explore conformational spaces, compute thermodynamic properties, and understand molecular behavior [23]. Traditional simulation methods often face limitations in scalability and efficiency [6], especially when dealing with the vast and intricate energy landscapes characteristic of systems described by Boltzmann distributions [7].

This thesis addresses these limitations by exploring and developing **better neural samplers for Boltzmann distribution**, aiming to enhance sampling efficacy. This involves exploring multiple avenues: initially, we investigate the development of a Multi-Step Diffusive KL (DiKL) Sampler, which employs a specialized training objective [11] and iterative refinement to target these distributions. Subsequently, and as a primary focus, we delve into methods inspired by Langevin dynamics—an approach inherently suited for sampling Boltzmann distributions given its direct use of energy gradients—to achieve superior sample quality and computational efficiency. Across these approaches, we investigate novel techniques that leverage the power of deep learning to adaptively learn the underlying dynamics of the sampling process, such as moving beyond fixed parameters to data-driven, position-dependent step sizes. This work is motivated by the critical need for improved sampling methodologies that can overcome the inherent challenges of high-dimensional Boltzmann distributions, enable better generalization, and provide reliable estimates for complex systems.

1.1 Contributions

This thesis presents several key contributions to the field of neural sampling and its applications:

- **Boltzmann Distribution & Neural Samplers:** We introduce the Boltzmann distribution, its role in molecular modeling, and associated sampling needs. Foundational sampling methods and neural samplers using diffusive reverse KL divergence are then covered.
- **Multi-Step DiKL Sampler:** We introduce an extension of the existing Diffusive KL sampler, reformulating it within a multi-step framework to enhance its sampling capacity and convergence properties.
- **Langevin Sampler with Learned Dynamics:** We propose a novel neural sampler, which integrates a learned, position-dependent step size into the foundational framework of Langevin sampling. This allows for adaptive and efficient exploration of complex probability distributions.
- **Metropolis-Adjusted Langevin Sampler with Learned Dynamics:** Building on the learned dynamics framework, we integrate adaptive Langevin steps with a Metropolis-Hastings acceptance mechanism. This hybrid approach targets rigorous convergence guarantees while maintaining sampling efficiency.

1.2 Outline

The remainder of this thesis is structured as follows:

- **Chapter 2** provides essential background information on Boltzmann distribution and introduces the foundational concepts of neural samplers relevant to this work.
- **Chapter 3** details the design and implementation of the multi-step DiKL sampler, presenting its theoretical underpinnings and experimental validation.
- **Chapter 4** introduces the Langevin sampler with learned dynamics, elaborating on its architecture, optimization strategies, and performance evaluations.
- **Chapter 5** presents the Metropolis-Adjusted Langevin sampler with learned dynamics, discussing its theoretical advantages and demonstrating its empirical effectiveness.
- **Chapter 6** concludes the thesis by summarizing the main findings, discussing their implications, and outlining potential directions for future research.

Chapter 2

Background

This chapter provides essential background information underpinning the methods and applications discussed in this thesis. We begin by formally introducing the Boltzmann distribution, a cornerstone of statistical mechanics, and discuss its critical role in various scientific domains, with a particular focus on its application in molecular modeling to understand system behavior. Subsequently, we delve into fundamental sampling methodologies designed to draw samples from such distributions, covering traditional approaches like Langevin Dynamics and the Metropolis-Adjusted Langevin Algorithm (MALA). Finally, we introduce the concept of neural samplers, including those trained with objectives like the diffusive reverse KL divergence, as advanced techniques for tackling these challenging sampling problems.

2.1 The Boltzmann Distribution and its Central Role in Molecular Modeling

The **Boltzmann distribution** is a cornerstone of statistical mechanics and plays a pivotal role in describing the equilibrium properties of physical systems. For a system that can exist in various states or configurations $\mathbf{x}$, each associated with an energy $E(\mathbf{x})$, the probability $p(\mathbf{x})$ of observing the system in a specific configuration $\mathbf{x}$ at a constant temperature T is given by:

$$p(\mathbf{x}) = \frac{1}{Z} e^{-E(\mathbf{x})/k_B T}, \quad (2.1)$$

where k_B is the Boltzmann constant, and Z is the partition function, defined as $Z = \sum_{\mathbf{x}'} e^{-E(\mathbf{x}')/k_B T}$ (for discrete states) or $Z = \int e^{-E(\mathbf{x}')/k_B T} d\mathbf{x}'$ (for continuous states). The partition function normalizes the distribution and encodes significant thermodynamic information about the system. The Boltzmann distribution is fundamental not only in physics and chemistry but also finds applications

in machine learning, for instance, in the context of Boltzmann machines [1] and energy-based models [27].

Understanding the macroscopic behavior of many systems hinges on sampling from the Boltzmann distribution, as system properties are typically determined by ensemble averages of observables $A(\mathbf{x})$. These averages are defined as:

$$\langle A \rangle = \sum_{\mathbf{x}} A(\mathbf{x})p(\mathbf{x}) \quad \text{or} \quad \langle A \rangle = \int A(\mathbf{x})p(\mathbf{x})d\mathbf{x}. \quad (2.2)$$

Direct evaluation of these sums or integrals, however, is typically unachievable in practice. The primary obstacles are the high dimensionality of the system’s state space $\mathbf{x}$ and the often intricate form of the energy function $E(\mathbf{x})$. As a result, computational sampling methods, which can effectively utilize unnormalized Boltzmann probabilities (proportional to $e^{-E(\mathbf{x})/k_B T}$) without requiring Z , are critically important [11].

2.1.1 Application: Molecular Modeling

A prime domain where sampling from the Boltzmann distribution is critical is **molecular modeling** [24]. This field encompasses a range of theoretical principles and computational techniques used to represent, simulate, and analyze the behavior of molecules and molecular systems. These systems can vary in scale and complexity, from individual small organic molecules to large biological macromolecules such as proteins [3] [2], DNA [30], and RNA [26], as well as complex materials and condensed-phase systems [28]. At its core, molecular modeling seeks to describe the structure of molecules, the interactions between their constituent atoms, and how these structures and interactions govern the system’s overall properties and dynamics. This is typically achieved by employing mathematical models of physics, ranging from classical mechanics (e.g., molecular mechanics using force fields [10] [8]) to quantum mechanics for more detailed electronic structure calculations.

The importance of molecular modeling is vast and continually growing across diverse scientific disciplines. In **drug discovery and development** [29] [14], it plays a pivotal role in identifying potential drug targets, designing novel therapeutic agents, and understanding drug-receptor interactions at an atomic level. Molecular modeling allows researchers to investigate systems with a level of detail that is often difficult or impossible to achieve through experimental means alone, and it serves as a powerful tool to interpret experimental data and formulate new hypotheses.

The **necessity for sampling from the Boltzmann distribution** [4] in molecular modeling arises fundamentally from the stochastic and dynamic nature of these systems, as well as the immense complexity of their potential state spaces, which are governed by their energies.

- **High-Dimensional Conformational Space:** Molecules, particularly large ones like proteins, can adopt a vast number of different spatial arrangements or conformations. The energy landscape $E(\mathbf{x})$ governing these conformations is typically rugged, featuring numerous local energy minima (corresponding to stable or metastable states) separated by energy barriers. Sampling aims to explore these conformations according to their Boltzmann probabilities.
- **Statistical Mechanics Foundation:** As stated, macroscopic properties of a molecular system (e.g., free energies, reaction rates, average structural features) are determined by an ensemble average over all accessible states, weighted by their respective Boltzmann probabilities (Equation 2.1).
- **Ergodicity and Ensemble Averages:** To compute these thermodynamic observables, it is necessary to generate a representative set of configurations from the Boltzmann ensemble. Effective sampling methods must be capable of ergodically exploring the significant regions of the conformational space (those with non-negligible Boltzmann weights) to ensure that calculated averages are reliable and converged.

In essence, sampling from the Boltzmann distribution is the computational engine that allows molecular modelers to bridge the gap between the microscopic details of molecular interactions and energies, and the macroscopic, experimentally observable properties and behaviors of molecular systems [20].

2.2 Sampling Methods

To address the challenges outlined in molecular modeling and other fields requiring the characterization of complex probability distributions, various sampling methods have been used. Firstly, we introduce traditional sampling methods that form the bedrock of computational statistical mechanics.

2.2.1 Langevin Sampling

Langevin sampling aims to generate samples from a target probability distribution $p_d(\mathbf{x})$. In its continuous-time formulation, the standard Langevin Stochastic Differential Equation (SDE) with a constant step size parameter $\gamma > 0$ is defined as:

$$d\mathbf{x}_t = \gamma \nabla \log p_d(\mathbf{x}_t) dt + \sqrt{2\gamma} d\mathbf{w}_t \quad (2.3)$$

where $\mathbf{x}_t \in \mathbb{R}^D$ represents the state of the system at time t , $\nabla \log p_d(\mathbf{x}_t)$ is the gradient of the log-density of the target distribution (the score function), and $\mathbf{w}_t$ is a standard Wiener process.

The evolution of the probability distribution $p(\mathbf{x}, t)$ of the process $\mathbf{x}_t$ is governed by the Fokker-Planck Equation (FPE):

$$\frac{\partial p(\mathbf{x}, t)}{\partial t} = -\nabla \cdot (\gamma p(\mathbf{x}, t) \nabla \log p_d(\mathbf{x})) + \nabla^2(\gamma p(\mathbf{x}, t)) \quad (2.4)$$

Here, the term $\nabla^2(\cdot)$ denotes the Laplacian operator. If γ is constant, the second term becomes $\gamma \nabla^2 p(\mathbf{x}, t)$.

A key property of this SDE is its ability to converge to the target distribution $p_d(\mathbf{x})$. The stationary distribution, denoted $p(\mathbf{x})^*$, is achieved when the time derivative of the probability distribution is zero, i.e., $\frac{\partial p(\mathbf{x}, t)^*}{\partial t} = 0$. For the standard Langevin SDE, this condition implies:

$$-\nabla \cdot (\gamma p(\mathbf{x})^* \nabla \log p_d(\mathbf{x})) + \gamma \nabla^2 p(\mathbf{x})^* = 0 \quad (2.5)$$

This equation simplifies to:

$$\nabla \cdot (\gamma p(\mathbf{x})^* \nabla \log p_d(\mathbf{x}) - \gamma \nabla p(\mathbf{x})^*) = 0 \quad (2.6)$$

A sufficient condition for this to hold is that the term inside the divergence is zero:

$$\gamma p(\mathbf{x})^* \nabla \log p_d(\mathbf{x}) = \gamma \nabla p(\mathbf{x})^* \quad (2.7)$$

Dividing by $\gamma p(\mathbf{x})^*$ (assuming $\gamma \neq 0$ and $p(\mathbf{x})^* \neq 0$), we obtain:

$$\nabla \log p_d(\mathbf{x}) = \frac{\nabla p(\mathbf{x})^*}{p(\mathbf{x})^*} = \nabla \log p(\mathbf{x})^* \quad (2.8)$$

This equality implies that $p(\mathbf{x})^* = C \cdot p_d(\mathbf{x})$, where C is a normalization constant. Since both are probability distributions, $C = 1$, and thus $p(\mathbf{x})^* = p_d(\mathbf{x})$. Therefore, the stationary distribution of the continuous-time Langevin SDE is indeed the target distribution.

In practice, the SDE is discretized, often using the Euler-Maruyama method, leading to the Unadjusted Langevin Algorithm (ULA):

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \gamma \nabla \log p_d(\mathbf{x}_k) + \sqrt{2\gamma} \mathbf{z}_k \quad (2.9)$$

where $\mathbf{z}_k \sim \mathcal{N}(0, \mathbf{I})$. This discretization introduces an error, and the stationary distribution of ULA is biased. However, this bias is of order $O(\gamma)$, meaning that for a sufficiently small step size γ , the sampled distribution approximates the target distribution $p_d(\mathbf{x})$ reasonably well.

2.2.2 Metropolis-Adjusted Langevin Algorithm (MALA)

The Unadjusted Langevin Algorithm (ULA) introduces a bias due to the finite time step discretization of the continuous Langevin SDE. The **Metropolis-Adjusted Langevin Algorithm (MALA)** is a Markov Chain Monte Carlo (MCMC) method designed to correct this bias and ensure that the generated samples asymptotically converge to the exact target distribution $p_d(\mathbf{x})$. MALA achieves this by incorporating a Metropolis-Hastings acceptance-rejection step for proposals generated by a Langevin-like update.

The MALA procedure at each iteration k involves two main stages:

1. **Proposal Generation:** A candidate state $\mathbf{x}'$ is proposed from the current state $\mathbf{x}_k$ using a discretized Langevin step, with step size parameter $\gamma_m > 0$:

$$\mathbf{x}' = \mathbf{x}_k + \gamma_m \nabla \log p_d(\mathbf{x}_k) + \sqrt{2\gamma_m} \mathbf{z}_k \quad (2.10)$$

where $\mathbf{z}_k \sim \mathcal{N}(0, \mathbf{I})$. The probability density of proposing $\mathbf{x}'$ given $\mathbf{x}_k$, denoted by the proposal kernel $q(\mathbf{x}'|\mathbf{x}_k)$, is a Gaussian distribution:

$$q(\mathbf{x}'|\mathbf{x}_k) = \mathcal{N}(\mathbf{x}'; \mathbf{x}_k + \gamma_m \nabla \log p_d(\mathbf{x}_k), 2\gamma_m \mathbf{I}) \quad (2.11)$$

2. **Acceptance-Rejection Step:** The proposed state $\mathbf{x}'$ is accepted as the next state $\mathbf{x}_{k+1}$ with an acceptance probability $\alpha(\mathbf{x}', \mathbf{x}_k)$ defined by the Metropolis-Hastings criterion:

$$\alpha(\mathbf{x}', \mathbf{x}_k) = \min \left(1, \frac{p_d(\mathbf{x}')q(\mathbf{x}_k|\mathbf{x}')}{p_d(\mathbf{x}_k)q(\mathbf{x}'|\mathbf{x}_k)} \right) \quad (2.12)$$

If the proposal is accepted, $\mathbf{x}_{k+1} = \mathbf{x}'$. Otherwise, the chain remains at the current state, i.e., $\mathbf{x}_{k+1} = \mathbf{x}_k$. The term $q(\mathbf{x}_k|\mathbf{x}')$ is the probability density of proposing $\mathbf{x}_k$ from $\mathbf{x}'$:

$$q(\mathbf{x}_k|\mathbf{x}') = \mathcal{N}(\mathbf{x}_k; \mathbf{x}' + \gamma_m \nabla \log p_d(\mathbf{x}'), 2\gamma_m \mathbf{I}) \quad (2.13)$$

The MALA algorithm is guaranteed to sample from $p_d(\mathbf{x})$ because the resulting Markov chain satisfies the **detailed balance condition** with respect to $p_d(\mathbf{x})$. The detailed balance condition states that for any two states $\mathbf{x}$ and $\mathbf{y}$:

$$p_d(\mathbf{x})T(\mathbf{y}|\mathbf{x}) = p_d(\mathbf{y})T(\mathbf{x}|\mathbf{y}) \quad (2.14)$$

where $T(\mathbf{y}|\mathbf{x})$ is the total transition probability from state $\mathbf{x}$ to state $\mathbf{y}$. This ensures that $p_d(\mathbf{x})$ is the unique stationary distribution of the Markov chain generated by MALA, provided the chain is ergodic.

MALA typically offers better performance than simpler methods like Random Walk Metropolis, especially in higher dimensions, because its proposals are guided by the gradient of the target distribution, leading to higher acceptance rates for well-chosen step sizes γ_m . However, it requires the computation of $\nabla \log p_d(\mathbf{x})$, and the choice of γ_m remains crucial for efficiency.

2.3 Neural Samplers

In recent years, **neural samplers** [5, 21] have emerged as a powerful class of generative models that leverage the expressive capabilities of neural networks to sample from complex, high-dimensional probability distributions. A neural sampler can be broadly defined as a computational system that employs one or more neural networks to learn and subsequently generate samples from a target probability distribution $p_d(\mathbf{x})$, or an approximation thereof [12]. These methods aim to overcome some of the limitations inherent in traditional sampling techniques [22] [9].

Formally, many neural samplers learn a parameterized transformation $f_\theta : \mathcal{Z} \rightarrow \mathcal{X}$, where $\mathcal{Z}$ is a latent space endowed with a simple, easy-to-sample prior distribution $p_Z(\mathbf{z})$ (e.g., a standard multivariate Gaussian $\mathcal{N}(0, \mathbf{I})$). The neural network f_θ , with parameters θ , maps a latent variable $\mathbf{z} \sim p_Z(\mathbf{z})$ to a sample $\mathbf{x} = f_\theta(\mathbf{z})$ in the target data space $\mathcal{X}$. The parameters θ are optimized such that the distribution of generated samples $p_\theta(\mathbf{x})$ closely matches the target distribution $p_d(\mathbf{x})$.

Neural samplers aim to offer several advantages over traditional methods:

- **Expressive Power:** Deep neural networks are highly expressive function approximators capable of modeling intricate, non-linear relationships and capturing the complex structure of high-dimensional data distributions that may be challenging for traditional models with fixed functional forms.
- **Amortized Sampling:** For many architectures (e.g., GANs, VAEs, normalizing flows), once the model f_θ is trained, generating new samples can be computationally very efficient, often requiring only a single forward pass from the latent space. This "amortized" sampling contrasts with MCMC methods, which generate samples iteratively and may require substantial computation per sample or long runs to decorrelate.
- **Scalability and Dimensionality:** Certain neural network architectures and training techniques are designed to scale effectively to high-dimensional problems, potentially mitigating the "curse of dimensionality" that can affect some traditional sampling methods.

Despite these advantages, neural samplers also present challenges, including complex training procedures, potential issues like mode collapse (where the model fails to capture all modes of the target distribution), difficulties in evaluating sample quality and convergence, and often significant

computational requirements for training. Nevertheless, their rapid development continues to push the boundaries of generative modeling and sampling.

2.3.1 Neural Samplers with Diffusive Reverse KL Divergence

While neural samplers offer significant advantages, their training requires carefully chosen objective functions. A common choice for training a neural sampler $p_\theta(\mathbf{x})$ to approximate a target distribution $p_d(\mathbf{x})$ is the reverse Kullback-Leibler (KL) divergence:

$$\text{KL}(p_\theta \| p_d) = \mathbb{E}_{\mathbf{x} \sim p_\theta(\mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x})}{p_d(\mathbf{x})} \right]. \quad (2.15)$$

However, minimizing this objective is known to exhibit "mode-seeking" behavior [6]. If the target distribution $p_d(\mathbf{x})$ is multi-modal, the learned distribution $p_\theta(\mathbf{x})$ tends to focus on one or a few modes of $p_d(\mathbf{x})$, underestimating the overall variance and failing to capture the full diversity of the target. This issue of mode collapse is particularly problematic when comprehensive coverage of the target distribution is crucial.

To address this limitation, an alternative objective known as the *diffusive KL (DiKL) divergence* [11] has been introduced. The core idea is to convolve both the model distribution $p_\theta(\mathbf{x})$ and the target distribution $p_d(\mathbf{x})$ with Gaussian diffusion kernels before computing the KL divergence. This convolution smooths the distributions, effectively promoting connectivity between modes and thereby reducing the propensity for mode collapse. While DiKL still defines a valid divergence, its modified landscape can provide significantly improved mode coverage capabilities.

Diffusive Reverse KL (DiKL)

Gaussian convolution [25] is a well-established technique for smoothing functions and can be particularly effective for bridging and merging isolated modes in a probability distribution. This smoothing effect can also simplify complex distributions by effectively reducing the number of apparent modes, akin to convexification through Gaussian smoothing in other contexts.

Inspired by these properties, the Diffusive KL (DiKL) divergence between $p_\theta(\mathbf{x})$ and $p_d(\mathbf{x})$ is defined as a weighted sum of KL divergences between their convolved versions:

$$\text{DiKL}(p_\theta \| p_d) = \sum_{i=1}^{N_k} w_i \cdot \text{KL}((p_\theta * k_i) \| (p_d * k_i)), \quad (2.16)$$

where $\{k_i\}_{i=1}^{N_k}$ is a set of Gaussian convolution kernels (typically with varying bandwidths), w_i are positive weights, and $*$ denotes the convolution operation. Let $p_{\theta,i}(\mathbf{x}) \triangleq (p_\theta * k_i)(\mathbf{x})$ and $p_{d,i}(\mathbf{x}) \triangleq (p_d * k_i)(\mathbf{x})$ denote the convolved distributions. The use of multiple kernels allows the

comparison of distributions at different scales, promoting gradual connectivity between modes.

Minimizing DiKL ensures that the neural sampler $p_\theta(\mathbf{x})$ approximates the target distribution $p_d(\mathbf{x})$ [31]. This can be understood by noting that if $\text{DiKL}(p_\theta||p_d) = 0$, then $\text{KL}(p_{\theta,i}||p_{d,i}) = 0$ for all i (since $w_i > 0$). This implies $p_{\theta,i}(\mathbf{x}) = p_{d,i}(\mathbf{x})$ for all i and $\mathbf{x}$. In terms of their Fourier transforms, denoted by $\mathcal{F}$, this means:

$$\begin{aligned} (p_\theta * k_i)(\mathbf{x}) &= (p_d * k_i)(\mathbf{x}) \Leftrightarrow \mathcal{F}\{p_\theta * k_i\}(\omega) = \mathcal{F}\{p_d * k_i\}(\omega) \quad \forall \omega \\ &\Leftrightarrow \mathcal{F}\{p_\theta\}(\omega) \cdot \mathcal{F}\{k_i\}(\omega) = \mathcal{F}\{p_d\}(\omega) \cdot \mathcal{F}\{k_i\}(\omega) \quad \forall \omega. \end{aligned} \quad (2.17)$$

Since the Fourier transform of a Gaussian kernel, $\mathcal{F}\{k_i\}(\omega)$, is non-zero for all frequencies ω , we can divide by it, yielding $\mathcal{F}\{p_\theta\}(\omega) = \mathcal{F}\{p_d\}(\omega)$. By the uniqueness of the Fourier transform, this implies $p_\theta(\mathbf{x}) = p_d(\mathbf{x})$. Thus, driving DiKL to zero effectively drives p_θ to match p_d .

The analytical gradient of a single component of the DiKL loss (for a specific kernel k_i) with respect to the parameters θ of the model p_θ can be derived. If samples $\mathbf{x}_i$ are drawn from $p_{\theta,i}(\mathbf{x})$ via a reparameterizable process $\mathbf{x}_i = g_\theta(\mathbf{z}, \epsilon_i)$ [15] (where $\mathbf{z}$ is base noise drawn from a simple distribution, ϵ_i is noise sampled from a distribution corresponding to kernel k_i , and g_θ incorporates the neural sampler f_θ that maps $\mathbf{z}$ to a "clean" sample, which is then perturbed by ϵ_i), the gradient is:

$$\begin{aligned} \nabla_\theta \text{KL}(p_{\theta,i}||p_{d,i}) &= \nabla_\theta \mathbb{E}_{\mathbf{x}_i \sim p_{\theta,i}} \left[\log \frac{p_{\theta,i}(\mathbf{x}_i)}{p_{d,i}(\mathbf{x}_i)} \right] \\ &= \mathbb{E}_{\mathbf{z}, \epsilon_i} \left[(\nabla_{\mathbf{x}_i} \log p_{\theta,i}(\mathbf{x}_i) - \nabla_{\mathbf{x}_i} \log p_{d,i}(\mathbf{x}_i)) \frac{\partial \mathbf{x}_i}{\partial \theta} \right], \end{aligned} \quad (2.18)$$

where $\mathbf{x}_i$ in the score terms is the sample generated via $g_\theta(\mathbf{z}, \epsilon_i)$, and $\frac{\partial \mathbf{x}_i}{\partial \theta}$ is the Jacobian of the sampling path with respect to θ . This gradient requires access to the score functions of the convolved distributions, $\nabla_{\mathbf{x}_i} \log p_{\theta,i}(\mathbf{x}_i)$ and $\nabla_{\mathbf{x}_i} \log p_{d,i}(\mathbf{x}_i)$ with $\nabla_{\mathbf{x}_i} \log p_{d,i}(\mathbf{x}_i)$ can be effectively calculated as in [11]

Denoising Score Identity (DSI)

To implement the DiKL loss gradient (Equation 2.18), we also need the score function of the convolved model distribution, $\nabla_{\mathbf{x}_i} \log p_{\theta,i}(\mathbf{x}_i)$. While this term can sometimes be derived analytically if $p_\theta(\mathbf{x})$ has a simple form, analytical solutions are often intractable for complex neural samplers. Moreover, empirical observations suggest that using a separate neural network to approximate this score term can lead to improved generalization and applicability across various target functions. This approximation technique is often based on the **Denoising Score Identity (DSI)** [27].

For any convolution kernel $k_i(\mathbf{x}_i|\mathbf{x})$ (which defines how a "clean" sample $\mathbf{x}$ from $p_\theta(\mathbf{x})$ is

perturbed to $\mathbf{x}_i$), the score of the convolved distribution $p_{\theta,i}(\mathbf{x}_i) = \int p_{\theta}(\mathbf{x})k_i(\mathbf{x}_i|\mathbf{x})d\mathbf{x}$ can be expressed as:

$$\nabla_{\mathbf{x}_i} \log p_{\theta,i}(\mathbf{x}_i) = \mathbb{E}_{\mathbf{x} \sim p_{\theta}(\mathbf{x}|\mathbf{x}_i)} [\nabla_{\mathbf{x}_i} \log k_i(\mathbf{x}_i|\mathbf{x})], \quad (2.19)$$

where $p_{\theta}(\mathbf{x}|\mathbf{x}_i) \propto k_i(\mathbf{x}_i|\mathbf{x})p_{\theta}(\mathbf{x})$ is the Bayesian posterior distribution of the "clean" sample $\mathbf{x}$ given the "noisy" (convolved) sample $\mathbf{x}_i$. This identity relates the score of the noisy data distribution to an expectation of the score of the noise distribution itself, under the posterior of the clean data.

Instead of directly computing this expectation (which can be hard due to the posterior $p_{\theta}(\mathbf{x}|\mathbf{x}_i)$), a (kernel- or noise-level-conditioned) score network $s_{\phi}(\mathbf{x}_i, i)$ is trained to approximate $\nabla_{\mathbf{x}_i} \log p_{\theta,i}(\mathbf{x}_i)$. This network, with parameters ϕ , is typically trained by minimizing a score matching objective. A common and practical form of this objective, derived from score matching principles, is:

$$L(\phi) = \sum_{i=1}^{N_k} \mathbb{E}_{\mathbf{x} \sim p_{\theta}(\mathbf{x})} \mathbb{E}_{\mathbf{x}_i \sim k_i(\cdot|\mathbf{x})} [\|s_{\phi}(\mathbf{x}_i, i) - \nabla_{\mathbf{x}_i} \log k_i(\mathbf{x}_i|\mathbf{x})\|_2^2]. \quad (2.20)$$

This loss function encourages the network $s_{\phi}(\mathbf{x}_i, i)$ to predict the score of the perturbation kernel $k_i(\mathbf{x}_i|\mathbf{x})$ (i.e., the gradient of the log-noise-conditional density with respect to the noisy data $\mathbf{x}_i$). The expectation is taken over clean samples $\mathbf{x}$ from the current model $p_{\theta}(\mathbf{x})$ and noisy samples $\mathbf{x}_i$ generated by applying the kernel k_i to $\mathbf{x}$. This objective avoids direct interaction with the intractable posterior $p_{\theta}(\mathbf{x}|\mathbf{x}_i)$.

The Denoising Score Identity and the associated gradient objective (Equation 2.18) are general results applicable for any differentiable convolution kernel k_i including cases where kernels only applied to p_{θ} . This generality makes DSI a versatile tool for estimating score functions in various contexts, including the training of DiKL-based neural samplers and other score-based generative models, and it will be a recurring technique in subsequent chapters of this thesis.

Chapter 3

Multi-Step Diffusive KL Divergence Sampler

Introduction

The preceding chapter presented Diffusive KL (DiKL) divergence as a state-of-the-art technique for addressing the mode-seeking behavior inherent in reverse KL divergence within neural samplers. Nevertheless, the efficacy of single-step DiKL approaches is constrained in their capacity to accurately approximate complex target distributions. This chapter introduces a multi-step DiKL framework designed to enhance sampling performance by iteratively refining the approximation. A quantitative evaluation of the generated sample quality will be conducted, giving due consideration to the associated computational expenditure. The primary objective is to investigate whether an incremental, multi-stage approach can overcome the limitations of single-pass DiKL methods.

3.1 Methods

To augment the representational capacity of DiKL-based neural samplers, a conceptually straightforward strategy involves the sequential training of supplementary DiKL models. Each additional model is tasked with fitting the residual error, or the discrepancy, between the output of the preceding combined model and the target distribution. Within this multi-step framework, the loss function for the m -th step, denoted as $\text{m_DiKL}_{\mathcal{K}}(p_{\theta,m}^* \| q)$, is defined as:

$$\text{m_DiKL}_{\mathcal{K}}(p_{\theta,m}^* \| q) \triangleq \sum_{t=1}^T w(t) \text{KL}(p_{\theta,m} * k_t \| p_d * k_t) \quad (3.1)$$

$$= \sum_{t=1}^T w(t) \int p_{\theta,m}(x_t) (\log p_{\theta,m}(x_t) - \log p_d(x_t)) dx_t \quad (3.2)$$

with the push-forward probability $p_{\theta,m}$ obtained from an initial distribution p_0 through the transformation $f_{\theta,m}^* (p_0 \xrightarrow{f_{\theta,m}^*} p_{\theta,m})$. The composite neural sampler at step m , $f_{\theta,m}^*(x)$, is an additive aggregation of previously trained, fixed samplers and the currently trained sampler:

$$f_{\theta,m}^*(x) = f_1(x) + f_2(x) + \cdots + f_{\theta,m-1}(x) + f_{\theta,m}(x) \quad (3.3)$$

Here, $w(t)$ represents a positive scalar weighting function, and $\mathcal{K} = \{k_1, \dots, k_T\}$ denotes a set of (scaled) Gaussian convolution kernels, where each $k_t(x_t | x) = \mathcal{N}(x_t | \alpha_t x, \sigma_t^2 I)$. The distribution $p_{\theta,m}(x)$ is the push-forward probability density resulting from the application of the composite neural sampler $f_{\theta,m}^*(x)$. In this formulation, $f_i(x)$ for $i < m$ are the fixed and pre-trained DiKL neural samplers from the i^{th} step, and only the parameters of the current step’s sampler, $f_{\theta,m}(x)$, are subject to training.

The analytical gradient of the loss function (Equation 3.1) with respect to the parameters θ of $f_{\theta,m}(x)$, for a specific kernel k_t , retains the same form as in the single-step DiKL framework:

$$\nabla_{\theta} \text{DiKL}_{k_t}(p_{\theta,m} || p_d) = \nabla_{\theta} \text{KL}(p_{\theta,m} * k_t || p_d * k_t) \quad (3.4)$$

$$= \int p_{\theta}(x_t) (\nabla_{x_t} \log p_{\theta,m}(x_t) - \nabla_{x_t} \log p_d(x_t)) \frac{\partial x_t}{\partial \theta} dx_t \quad (3.5)$$

To potentially enhance model capacity and avoid direct analytical computation, the score term of the model distribution, $\nabla_{x_t} \log p_{\theta,m}(x_t)$, can be approximated using the Denoising Score Identity. Concurrently, the score of the target distribution, $\nabla_{x_t} \log p_d(x_t)$, is approximated as in [11].

3.2 Experiments and Results

This section details the empirical evaluation of the proposed multi-step DiKL approach. We introduce two benchmark datasets, adapted from the original DiKL publication, which serve as robust proxies for complex, real-world energy functions frequently encountered in domains such as molecular modelling. Furthermore, a justification for the chosen strategy of fitting residuals (additive refinement) over sequential function composition is provided. Finally, the performance of the multi-step DiKL neural samplers is presented, accompanied by an analysis of their respective advantages and inherent limitations.

3.2.1 Experiment Setting

The quality of the generated samples is assessed using two distinct datasets: MOG-40 and MW-32.

MOG-40 (Mixture of Gaussians) The MOG-40 dataset, a mixture of 40 two-dimensional Gaussian distributions (Figure 3.1), serves as a standard benchmark. This dataset effectively models the multi-modal probability landscapes often encountered when dealing with Boltzmann distributions, where multiple high-probability (low-energy) regions correspond to dominant system states. Each Gaussian mode can be seen as representing a significant region of probability mass. A key advantage of MOG-40 is its known target density, enabling precise quantitative evaluation of sample quality. Critically, it allows assessment of a sampler’s ability to achieve comprehensive mode coverage—an indispensable attribute for accurately sampling complex Boltzmann distributions, such as those found in molecular modeling applications.

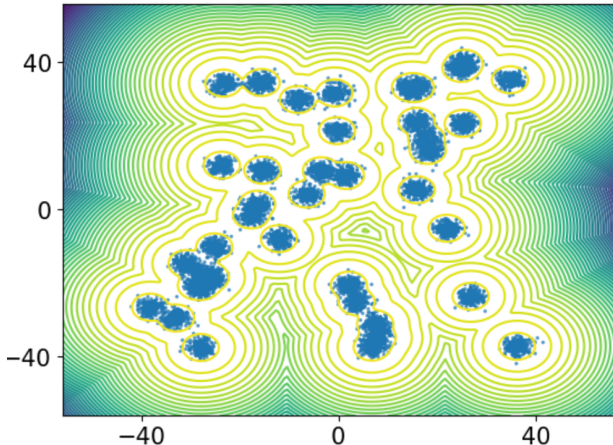


Figure 3.1: Ground truth of MOG40 data.

MW-32 (Many-Wall) The MW-32 dataset [19] presents a significantly more challenging scenario for sampling complex Boltzmann distributions. It is constructed by stacking 32 instances of a 2D Double-Well potential, culminating in a distribution with 2^{32} distinct modes (Figure 3.2). These modes are associated with varying weights, thereby stringently testing a sampler’s capability not only to identify all modes but also to accurately capture their respective probabilities. In the context of experiments with this dataset, the neural sampler functions as a Boltzmann Generator [21], tasked with generating samples from n-body systems where the energy is defined by pairwise distances between particles. Such an energy definition gives rise to a complex Boltzmann distribution relevant to various physical systems, including those studied in molecular modeling [18]. Owing to the high dimensionality of the MW-32 target distribution, the evaluation primarily focuses on its 2D marginal distributions. Specifically, pairs from the first four dimensions are examined: 1st and 3rd dimensions; 2nd and 4th dimensions; 1st and 2nd dimensions; and 3rd and 4th dimensions.

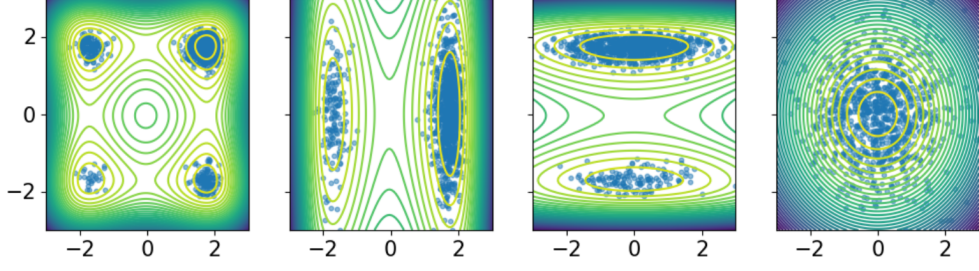


Figure 3.2: Ground truth of MW32 data.

For both datasets, the performance of the neural sampler is quantified using the **expected log-likelihood**, $E[\log p_d(x)]$. In this metric, x represents samples drawn from the neural sampler, and $p_d(x)$ is the probability density function of the target distribution. Higher values of this metric indicate a better approximation of the target distribution by the sampler.

3.2.2 Analysis

Multi-Step Configuration: Additive Residual Fitting

In the proposed multi-step DiKL framework, subsequent models are trained to fit the residual error of the preceding stages. This additive refinement is represented as $f_m^*(x) = f_{m-1}^*(x) + f_{\theta,m}(x)$, where $f_{m-1}^*(x)$ encapsulates the collective output of the $(m-1)$ previous steps (i.e., $\sum_{i=1}^{m-1} f_i(x)$), and $f_{\theta,m}(x)$ is the new model being trained at the m -th step. This additive approach is deliberately chosen over a sequential (or compositional) application, such as $f_m^*(x) = f_{\theta,m}(f_{m-1}^*(x))$.

The primary rationale for this choice stems from the empirical observation that DiKL models, regardless of their input data distribution, tend to initialize their mapping transformations from a near-central point. This behavior is attributed to the common practice of initializing model weights with small values [13]. Consequently, if a new model were applied sequentially to the output of a previous one, it would likely expend considerable training time and computational resources attempting to learn an approximate identity mapping before it could begin to effectively refine the sample distribution. Figure 3.3 illustrates this phenomenon, where a sequentially applied model consistently initiates its learning process from a similar, often suboptimal, starting configuration. By contrast, the strategy of fitting the residual allows the new model $f_{\theta,m}(x)$ to directly focus on correcting the errors of $f_{m-1}^*(x)$, thereby channeling its capacity towards improving sample quality from the outset without the prerequisite of learning an identity transformation.

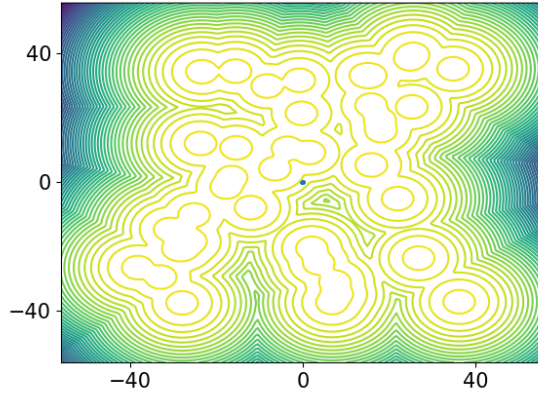


Figure 3.3: Due to model initialization, for multi-step in sequential cases, the process often starts from this point, indicating a challenge in learning identity mappings efficiently.

Performance on MOG-40

Figures 3.4a, 3.4b, and 3.4c present the samples generated by 1-step, 2-step, and 3-step DiKL models, respectively, when applied to the MOG-40 dataset. A visual inspection of these figures suggests a progressive improvement in the quality of the generated samples with an increasing number of steps; the distributions appear to cover the modes of the target more effectively.

This qualitative observation is quantitatively corroborated by the chosen evaluation metric, the expected log-likelihood, as depicted in Figure 3.5. The plot demonstrates an increase in the metric with each additional step, signifying a closer approximation to the target distribution. However, it is noteworthy that while an improvement is evident, the magnitude of this improvement between successive steps is modest.

Furthermore, an important practical consideration is the computational cost. The training expenditure for each step in the multi-step DiKL process remains constant. This is because each new DiKL model ($f_{\theta,m}$) is trained independently from scratch (e.g., requiring approximately 2.5 hours per step on an NVIDIA H100 GPU for each DiKL training instance). The combination of modest performance gains and consistent per-step computational demands motivates the exploration of alternative strategies that could yield more substantial enhancements in sample quality or achieve comparable quality with greater computational efficiency.

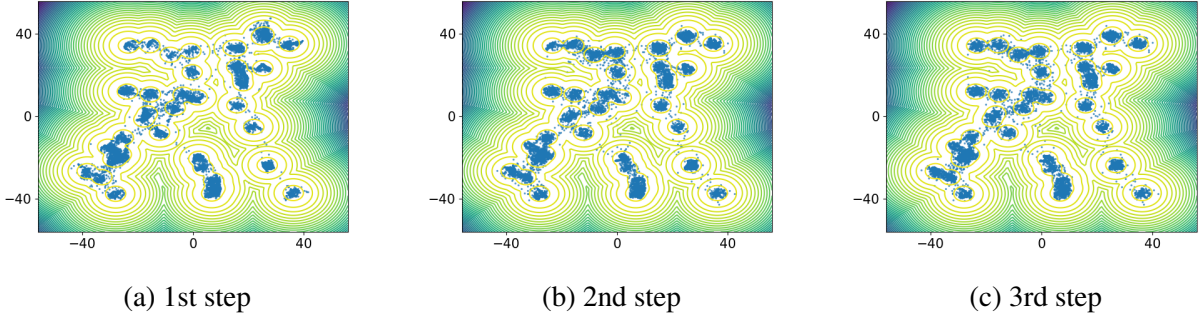


Figure 3.4: Generation results for 1-step, 2-step, and 3-step DiKL on MOG-40.

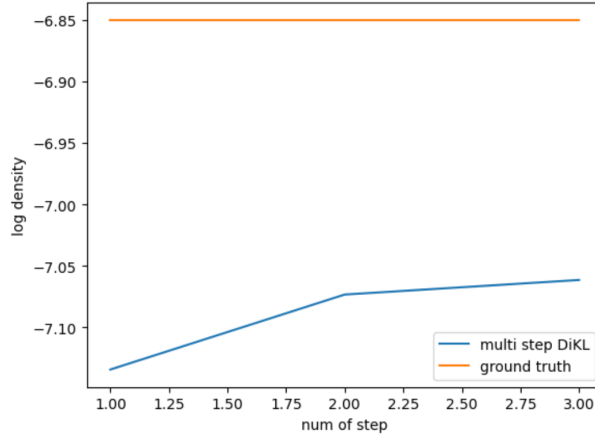


Figure 3.5: Metric change (expected log-likelihood) with the number of steps for DiKL on MOG-40.

Performance on MW-32

A comparable trend of marginal improvement in sample quality with an increasing number of steps is observed for the more complex MW-32 dataset. Figures 3.6, 3.7, and 3.8 illustrate the 2D marginal distributions of samples generated by 1-step, 2-step, and 3-step DiKL models, respectively. While visual enhancements in mode coverage and representation can be discerned with additional steps, these improvements are, akin to the MOG-40 case, limited in scale as in Figure 3.9. This outcome suggests that merely increasing the number of residual fitting steps may not be the most effective or efficient pathway to achieving high-fidelity samples for such intricate, high-dimensional distributions. The diminishing returns in quality improvement relative to the linear increase in computational cost underscore the need for more advanced techniques to tackle such challenging sampling problems.

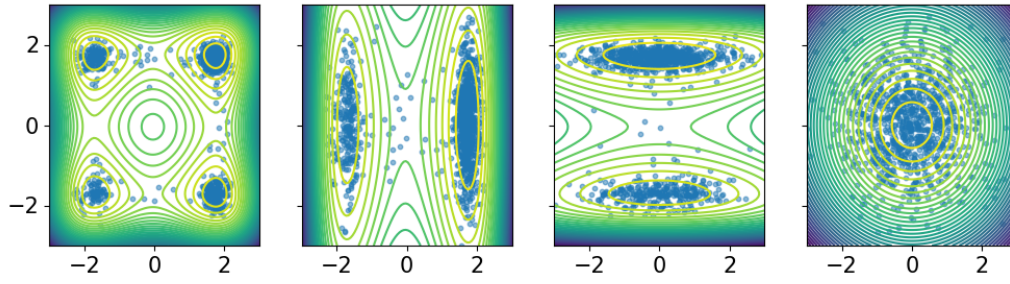


Figure 3.6: Generated samples for 1-step DiKL in MW32.

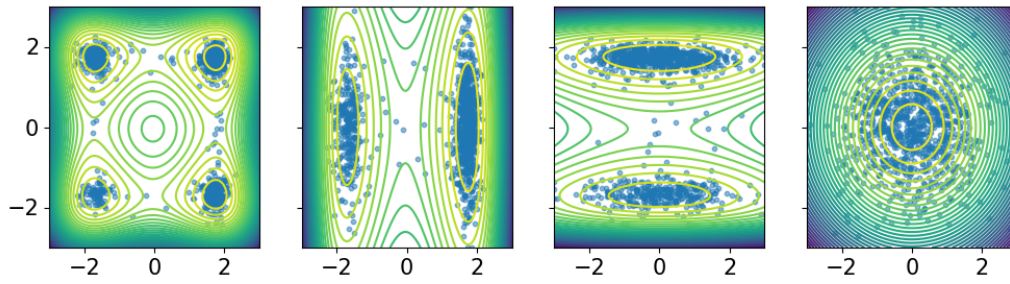


Figure 3.7: Generated samples for 2-step DiKL in MW32.

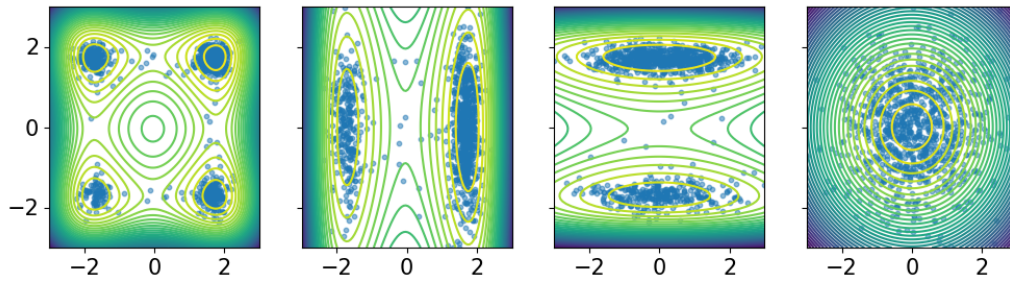


Figure 3.8: Generated samples for 3-step DiKL in MW32.

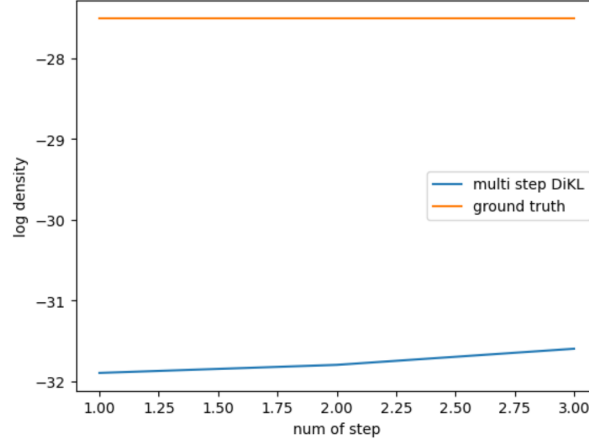


Figure 3.9: Metric change (expected log-likelihood) with the number of steps for DiKL on MW-32.

3.3 Summary

This chapter introduced and evaluated a multi-step Diffusive Inverse KL (DiKL) divergence framework. The core approach involves an iterative refinement of the sampling distribution by training successive DiKL models to fit the residual error of preceding stages, an additive method chosen to avoid inefficient learning of identity mappings. Each new model is trained using a DiKL loss, with gradient computation similar to single-step DiKL, and score terms estimable via Denoising Score Identity.

Empirical validation on benchmark datasets, MOG-40 and MW-32, demonstrated that while increasing the number of steps improved sample quality, these gains were modest. A significant drawback was the computational cost, which scaled linearly with each new DiKL model trained from scratch.

In conclusion, the multi-step DiKL approach offers incremental capacity enhancement over single-step methods but exhibits diminishing returns relative to its computational expense. This suggests that while iteratively correcting residuals is a valid concept, its direct application here may not be optimally efficient for complex distributions, motivating further research into more effective and scalable neural sampling strategies.

Chapter 4

Langevin Sampler with Learned Dynamics

Introduction

The preceding chapter on multi step DiKL illuminated a pathway to enhance neural samplers by iteratively fitting residual errors. While this approach demonstrated incremental improvements in sample quality, it also underscored inherent limitations: the gains per additional step were often modest, and the computational burden of training a distinct, complex neural sampler for each stage presented a significant challenge to efficiency and scalability.

This chapter transitions to investigate such an alternative paradigm: Langevin dynamics, augmented with learned, position-dependent characteristics. We explore whether this framework, by directly engaging with the score of the target distribution, $\nabla_x \log p_d(x)$, and by employing adaptive mechanisms for guiding the sampling process, can surmount some of the efficacy and efficiency hurdles encountered with the multi-step DiKL approach. The central hypothesis is that a more nuanced, physics-informed iterative refinement, as offered by Langevin-based methods, may lead to more substantial improvements in approximating complex target distributions.

Specifically, this chapter details strategies for learning a position-dependent step-size function, $\gamma(x)$ to tailor the Langevin updates to the local geometry of the data manifold. Furthermore, we will examine iterative sampling schemes, including multi-step Langevin processes with sequentially trained components and, crucially, auto-regressive Langevin sampling. The latter, which involves the repeated application of a single, highly capable learned sampler, offers a particularly promising avenue to enhance sample quality without the linear accumulation of distinct models.

Beyond direct generation, the chapter also considers the role of learned Langevin samplers as powerful post-processors, capable of refining samples generated by other samplers. A rigorous analysis of the inherent biases associated with position-dependent step sizes in Langevin SDEs, and the computational implications of their correction, will also be presented to provide a complete theoretical picture.

4.1 Methods for Learned Langevin Dynamics

This section details a strategy for adaptively learning a position-dependent step size, denoted $\gamma(x)$, for Langevin dynamics. The primary application is the refinement of samples, initiated from a distribution $p(x)$ (often pure noise), to better align with a target distribution $p_d(x)$ through a single Langevin update. The function $\gamma(x)$ is typically represented by a neural network whose parameters are learned.

The core of this method lies in optimizing a one-step Langevin transition kernel:

$$p_\gamma(y|x) = \mathcal{N}(y; x + \gamma(x)\nabla \log p_d(x), 2\gamma(x)I) \quad (4.1)$$

where $x \sim p(x)$ is an initial sample, and y is the refined sample. The step-size function $\gamma(x)$ is learned by minimizing the Kullback-Leibler (KL) divergence between the distribution of refined samples, $p_\gamma(y)$, and the target distribution $p_d(y)$:

$$\gamma^*(\cdot) = \arg \min_{\gamma(\cdot)} \mathbb{D}_{\text{KL}}(p_\gamma \| p_d) \quad (4.2)$$

where $p_\gamma(y) = \int p_\gamma(y|x)p(x)dx$.

The gradient of this KL divergence with respect to the parameters governing $\gamma(x)$ (for simplicity, denoted ∇_γ) can be expressed as:

$$\nabla_\gamma \mathbb{D}_{\text{KL}}(p_\gamma \| p_d) = E_{x \sim p(x)} E_{y \sim p_\gamma(y|x)} \left[(\nabla_y \log p_\gamma(y) - \nabla_y \log p_d(y)) \frac{\partial y}{\partial \gamma} \right] \quad (4.3)$$

In this expression, $\nabla_y \log p_d(y)$ is the score of the target distribution (assumed accessible), and $\frac{\partial y}{\partial \gamma(x)}$ can be obtained via automatic differentiation of the Langevin update step $y = x + \gamma(x)\nabla \log p_d(x) + \sqrt{2\gamma(x)}Z$, where $Z \sim \mathcal{N}(0, I)$. A critical challenge is the estimation of the score function $\nabla_y \log p_\gamma(y)$ for the distribution of the refined samples. Two primary approaches for estimating $\nabla_y \log p_\gamma(y)$ are considered:

Analytical Score Estimation via Bayesian Posterior The score $\nabla_y \log p_\gamma(y)$ can be derived analytically as an expectation over the Bayesian posterior $p(x|y) \propto p_\gamma(y|x)p(x)$:

$$\nabla_y \log p_\gamma(y) = \nabla_y \log \int \mathcal{N}(y|x + \gamma(x)\nabla \log p_d(x), 2\gamma(x)I)p(x)dx \quad (4.4)$$

$$= E_{p(x|y)} [\nabla_y \log \mathcal{N}(y|x + \gamma(x)\nabla \log p_d(x), 2\gamma(x)I)] \quad (4.5)$$

$$= E_{p(x|y)} \left[\frac{x + \gamma(x)\nabla \log p_d(x) - y}{2\gamma(x)} \right] \quad (4.6)$$

Sampling from $p(x|y)$ can be performed using techniques like MALA. The unnormalized log-posterior is $\log p(x) + \log p_\gamma(y|x)$. However, MALA is known for its significant computational expense, potentially limiting the practicality of this approach.

Score Estimation via Denoising Score Matching Analogy An alternative, potentially more efficient method draws inspiration from Denoising Score Identity (DSI) and related techniques (e.g., as utilized in DiKL, discussed in Chapter 2). The fundamental expression for the score remains an expectation involving the score of the transition kernel:

$$\nabla_y \log p_\gamma(y) = E_{p(x|y)} [\nabla_y \log p_\gamma(y | x)] \quad (4.7)$$

where the score of the transition kernel, $\nabla_y \log p_\gamma(y | x) = -\frac{y - (x + \gamma(x)\nabla \log p_d(x))}{2\gamma(x)}$, is readily calculated. This approach differs from the first in how the expectation over $p(x|y)$ is handled. Instead of relying on full MALA, techniques related to Denoising Score Identity allow for the training of a time-conditioned score network $s_\phi(y)$ to approximate $\nabla_y \log p_\gamma(y)$. This is achieved by minimizing a score matching loss with respect to the parameters ϕ of the network s_ϕ :

$$\begin{aligned} L(\phi) &= \int \left\| s_\phi(y) - \int \nabla_y \log p(y | x) p_\gamma(x | y) dx \right\|_2^2 p_\gamma(y) dy \\ &= \iint \|s_\phi(y) - \nabla_y \log p_\gamma(y | x)\|_2^2 p_\gamma(x, y) dx dy + \text{const.} \end{aligned} \quad (4.8)$$

This formulation effectively bypasses the direct sampling from $p(x|y)$ for each gradient step, often leading to a more computationally tractable training procedure.

4.1.1 Multi-Step Langevin Sampling for Enhanced Generative Capacity

The inherent capacity of a Langevin sampler to generate complex data distributions can be significantly enhanced through the implementation of a multi-step approach. This strategy mirrors the conventional practice in standard Langevin dynamics, where iterative updates are fundamental to achieving convergence to the target distribution. In traditional Langevin sampling with a constant step size, the theoretical convergence rate is typically characterized by $O(1/T)$ or $O(1/\sqrt{T})$, where T represents the number of steps. This rate signifies that an increased number of steps generally leads to a more accurate approximation of the target distribution.

Given this established relationship, it is logical to extend the multi-step paradigm to the learned Langevin sampler framework. To operationalize this multi-step approach, a sequential chaining of trained samplers is employed. The output, or more precisely, the samples generated by an initial trained sampler, serve as the input for a subsequent sampler in the sequence. This process can be

iterated to form a chain of arbitrary length. Formally, let $p_d(y)$ denote the target data distribution. If we consider a two-step process, the first sampler, parameterized by a learned step-size function $\gamma_1(x)$, generates samples y_1 from its learned distribution $p_{\gamma_1}(x)$. These samples y_1 then become the starting points for the second sampler, parameterized by $\gamma_2(y_1)$. Crucially, when training the second model (parameterized by γ_2), the parameters of the first model (governing γ_1) are typically frozen. This ensures that the first stage of sampling remains consistent, providing a stable input distribution for the subsequent refinement steps. The conditional distribution for the second step, $p_{\gamma_2|\gamma_1}(y_2|y_1)$, is defined as:

$$p_{\gamma_2|\gamma_1}(y_2|y_1) = \mathcal{N}(y_2; y_1 + \gamma_2(y_1)\nabla \log p_d(y_1), 2\gamma_2(y_1)I) \quad (4.9)$$

Here, $y_1 \sim p_{\gamma_1}(x)$ signifies that y_1 is a sample drawn from the distribution learned by the first sampler. The learnable step size $\gamma_2(y_1)$ is now a function of its input y_1 , allowing for adaptive movement within the sample space. This chaining mechanism effectively allows each subsequent sampler to refine the samples produced by its predecessor, incrementally pushing them closer to the true data distribution $p_d(y)$. This iterative refinement through multiple learned steps contributes to a more accurate and robust generative model.

4.1.2 Auto-Regressive Langevin Sampling for Sequential Refinement

While the multi-step approach involving distinct, sequentially trained samplers offers significant advantages in terms of capacity, an alternative and potentially more parsimonious strategy is the adoption of an auto-regressive Langevin sampler. This paradigm draws inspiration from auto-regressive models prevalent in sequence modeling, where a single model iteratively processes its own output to generate subsequent elements in a sequence. In the context of Langevin sampling, this translates to recursively applying the *same* trained model for multiple steps of refinement.

This approach offers a compelling advantage in terms of model economy, as it obviates the need to train, store, and manage multiple distinct models, each with its own set of parameters. Instead, a single, highly capable model learns to progressively refine samples, leveraging its internal representations and learned dynamics across multiple iterations.

The underlying rationale is that a well-trained Langevin sampler, characterized by its learned step-size function $\gamma(x)$ and an accurate approximation of the target score (or the model score $\nabla_y \log p_\gamma(y)$), encapsulates a deep understanding of the dynamics required to move from an initial state towards the target distribution. By repeatedly applying this singular model, each iteration builds upon the refinement achieved in the previous step, allowing the sampler to gradually converge to the desired data distribution. This approach is particularly effective when the learned score function and step-size function are sufficiently accurate and generalized, allowing the single model to effectively navigate the entire sampling process from initial noise to realistic data samples.

4.1.3 Bias Analysis of Position-Dependent Step Sizes

This section analyzes the convergence properties of Langevin sampling, particularly when employing a position-dependent step size, $\gamma(\mathbf{x}_t)$, which is a function of the current position $\mathbf{x}_t$. The Stochastic Differential Equation (SDE) governing the dynamics is:

$$d\mathbf{x}_t = \gamma(\mathbf{x}_t) \nabla \log p_d(\mathbf{x}_t) dt + \sqrt{2\gamma(\mathbf{x}_t)} d\mathbf{w}_t \quad (4.10)$$

The corresponding Fokker-Planck Equation (FPE) for this SDE, with a position-dependent diffusion coefficient $D(x) = \gamma(x)$, describes the evolution of the probability density $p(x, t)$:

$$\frac{\partial p(x, t)}{\partial t} = -\nabla \cdot (p(x, t) \gamma(x) \nabla \log p_d(x)) + \nabla^2 (\gamma(x) p(x, t)) \quad (4.11)$$

For the stationary distribution $p(x)^*$, where $\frac{\partial p(x, t)^*}{\partial t} = 0$, the FPE simplifies to:

$$-\nabla \cdot (p(x)^* \gamma(x) \nabla \log p_d(x)) + \nabla \cdot \nabla (\gamma(x) p(x)^*) = 0 \quad (4.12)$$

This implies that, for a stationary state to be reached, the net probability flux must be zero. A sufficient condition for this is:

$$p(x)^* \gamma(x) \nabla \log p_d(x) = \nabla (\gamma(x) p(x)^*) \quad (4.13)$$

Expanding the right-hand side using the product rule for gradients ($\nabla(fg) = f\nabla g + g\nabla f$):

$$p(x)^* \gamma(x) \nabla \log p_d(x) = p(x)^* \nabla \gamma(x) + \gamma(x) \nabla p(x)^* \quad (4.14)$$

If we test whether the target distribution $p_d(x)$ is a stationary solution, i.e., substitute $p(x)^* = p_d(x)$, we obtain:

$$p_d(x) \gamma(x) \nabla \log p_d(x) = p_d(x) \nabla \gamma(x) + \gamma(x) \nabla p_d(x) \quad (4.15)$$

Since $\nabla \log p_d(x) = \frac{\nabla p_d(x)}{p_d(x)}$, the left side simplifies to $p_d(x) \gamma(x) \frac{\nabla p_d(x)}{p_d(x)} = \gamma(x) \nabla p_d(x)$. Thus, the condition becomes:

$$\gamma(x) \nabla p_d(x) = p_d(x) \nabla \gamma(x) + \gamma(x) \nabla p_d(x) \quad (4.16)$$

This equation holds if and only if $p_d(x) \nabla \gamma(x) = 0$. For this to be true for all x where $p_d(x) \neq 0$, it must be that $\nabla \gamma(x) = 0$, implying that $\gamma(x)$ is a constant.

However, if $\gamma(x)$ is not constant (i.e., $\nabla \gamma(x) \neq 0$), then $p_d(x) \nabla \gamma(x) \neq 0$ for some x . Consequently, the stationary distribution $p(x)^*$ of the SDE (Equation 4.10) with a position-dependent step size $\gamma(x)$ is **no longer guaranteed to be the target distribution** $p_d(x)$. This discrepancy

introduces a systematic bias that is inherent to the continuous-time process itself, even before considering discretization errors. This bias is not necessarily of order $O(\gamma)$ and can be substantial, potentially leading to significant deviations from the desired target distribution.

To recover $p_d(x)$ as the stationary distribution when using a position-dependent step size $\gamma(x)$, the drift term of the SDE must be modified. We introduce an additional correction term $b(\mathbf{x}_t)$ into the SDE:

$$d\mathbf{x}_t = (\gamma(\mathbf{x}_t)\nabla \log p_d(\mathbf{x}_t) + b(\mathbf{x}_t)) dt + \sqrt{2\gamma(\mathbf{x}_t)} d\mathbf{w}_t \quad (4.17)$$

The stationary condition for the associated FPE, assuming $p(x)^* = p_d(x)$, now requires (from Equation 4.13, replacing the original drift):

$$p_d(x)(\gamma(x)\nabla \log p_d(x) + b(x)) = \nabla(\gamma(x)p_d(x)) \quad (4.18)$$

Expanding the right side yields:

$$p_d(x)\gamma(x)\nabla \log p_d(x) + p_d(x)b(x) = \gamma(x)\nabla p_d(x) + p_d(x)\nabla \gamma(x) \quad (4.19)$$

Substituting $\gamma(x)\nabla p_d(x) = p_d(x)\gamma(x)\nabla \log p_d(x)$:

$$p_d(x)\gamma(x)\nabla \log p_d(x) + p_d(x)b(x) = p_d(x)\gamma(x)\nabla \log p_d(x) + p_d(x)\nabla \gamma(x) \quad (4.20)$$

This simplifies to $p_d(x)b(x) = p_d(x)\nabla \gamma(x)$, which implies that the required correction term is $b(x) = \nabla \gamma(x)$, assuming $p_d(x) \neq 0$. The corrected SDE that admits $p_d(x)$ as its stationary distribution is therefore:

$$d\mathbf{x}_t = (\gamma(\mathbf{x}_t)\nabla \log p_d(\mathbf{x}_t) + \nabla \gamma(\mathbf{x}_t)) dt + \sqrt{2\gamma(\mathbf{x}_t)} d\mathbf{w}_t \quad (4.21)$$

The Euler-Maruyama discretization of this corrected SDE yields the following transition kernel:

$$p_\gamma(y|x) = \mathcal{N}(y; x + \Delta t(\gamma(x)\nabla \log p_d(x) + \nabla \gamma(x)), 2\Delta t\gamma(x)\mathbf{I}) \quad (4.22)$$

(Assuming $\Delta t = 1$ for consistency with the initial kernel if it's a one-step update parameter).

Although incorporating the $\nabla \gamma(x)$ term theoretically ensures that the continuous-time Stochastic Differential Equation (SDE) converges to the intended target distribution, its practical application encounters substantial computational difficulties, particularly when $\gamma(x)$ is parameterized by a data-learned neural network, denoted $\gamma(x; \theta)$.

Implementing the theoretically exact correction term, $\nabla_x \gamma(x; \theta)$, for the adaptive step-size Langevin SDE presents substantial computational challenges. Both sampling with this term

and training its parameters θ necessitate evaluating complex derivatives. This includes second-order derivatives of $\gamma(x; \theta)$ itself (e.g., $\nabla_{\theta} \nabla_x \gamma(x; \theta)$). Furthermore, if γ depends on the target score $\nabla_x \log p_d(x)$, its spatial gradient $\nabla_x \gamma$ will involve the Hessian of the target log-probability, $\nabla_x^2 \log p_d(x)$, which is often intractable for complex, high-dimensional distributions.

These intensive computational requirements typically render the fully corrected SDE impractical for many models. Even so, we still **employ this corrected approach in specific instances within this work**, to observe its theoretical effects on issues such as bias and mode collapse, despite the associated costs. For most other scenarios, the potentially biased Unadjusted Langevin Algorithm (ULA) with a learned position-dependent step size remains the more feasible option.

4.1.4 Langevin Samplers as Post-Processors for Enhanced Sampling in Complex Systems

While our discussion has primarily focused on utilizing Langevin samplers to generate new data samples from an initial noise distribution, transforming unstructured input into meaningful outputs, an equally vital application lies in their role as post-processors. In this capacity, Langevin samplers are employed to refine or improve samples that have been previously obtained through other, potentially less computationally intensive, generative methods. This post-processing capability is especially crucial in scientific domains dealing with highly complex systems, such as molecular modeling, where accurately sampling from a desired equilibrium distribution (e.g., the Boltzmann distribution of molecular conformations) is a notoriously challenging computational task. Even advanced generative sampling techniques can sometimes produce samples that are not perfectly representative of the true underlying distribution or may lack fine-grained accuracy.

This post-processing step is also valuable for ensuring the transferability and robustness of results obtained from generative models [16] [17]. If a generative model is trained on one specific type of molecular system and then applied to a new, unseen molecule or a different thermodynamic condition, its initial generated samples might exhibit reduced precision. A Langevin-based post-processor can then help bridge this gap by refining these newly generated samples, adapting them more closely to the specific characteristics of the new system’s energy landscape or target distribution. By leveraging the local optimization capabilities inherent in Langevin dynamics, this refinement stage enhances the accuracy and reliability of the samples, allowing for more confident downstream predictions and better generalization across different systems or conditions. In essence, it provides a crucial final step to ensure that even rapidly generated or transferred samples attain the highest possible quality necessary for rigorous scientific analysis and interpretation.

4.2 Experiments and Results

In this section, we evaluate the performance of the learned Langevin sampler on the MOG-40 and MW-32 datasets to be consistent with method of multi-step DiKL (detailed in Chapter 3 section 3.2.1). We conduct a comprehensive study of the methodologies presented, including an analysis of the impact of step size dimensionality and the effectiveness of multi-step and auto-regressive sampling strategies.

4.2.1 Experiment Setting

Initial Sample Distributions For the MOG-40 dataset, initial samples for the Langevin dynamics are drawn from pure isotropic Gaussian noise, as illustrated in Figure 4.1. This setup tests the sampler’s ability to generate structured data from a completely unstructured starting point.

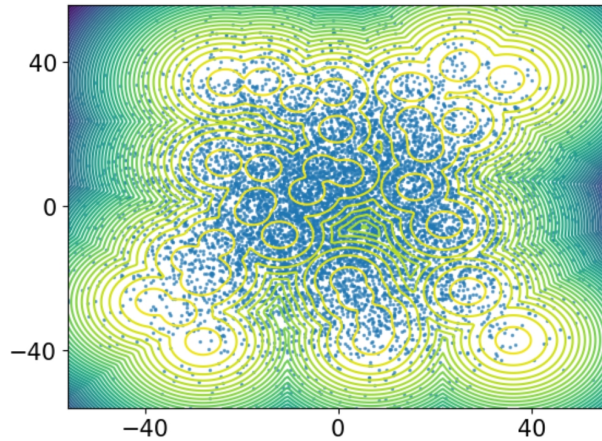


Figure 4.1: Initial pure noise distribution used as input for Langevin sampling on the MOG-40 dataset.

For the more complex MW-32 dataset, the Langevin sampler is primarily employed as a post-processor, with an emphasis on transfer learning scenarios. In this context, the initial samples are not pure noise. Instead, they are derived from true target samples to which a significant amount of noise has been added. This simulates a situation where an initial generative model produces approximate samples that require further refinement, as depicted in Figure 4.2. .

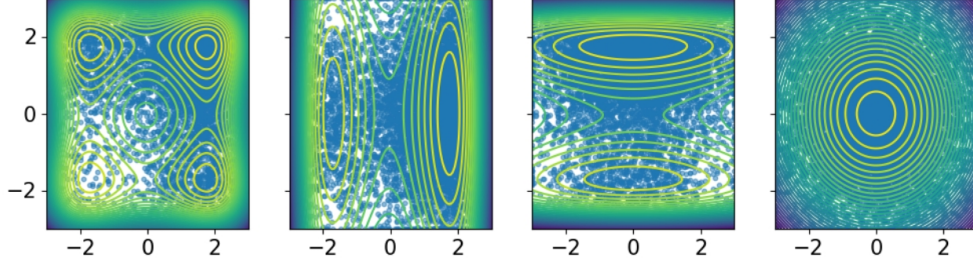


Figure 4.2: Initial samples for the MW-32 dataset, created by adding noise to target samples, simulating a post-processing or transfer learning scenario.

Model Architecture The neural network parameterizing the position-dependent step size, $\gamma(x)$, (referred to as the "stepsize net") is modeled as a Multi-Layer Perceptron (MLP). Its final layer employs a sigmoid activation function, scaled to ensure the step size remains within a predefined range: $y = \text{min_stepsize} + (\text{max_stepsize} - \text{min_stepsize}) \cdot \text{torch.sigmoid}(x_{\text{MLP}})$, where x_{MLP} is the MLP's output before the sigmoid. The score model, $s_\phi(y)$, used for approximating $\nabla_y \log p_\gamma(y)$ when the DSM-inspired approach is employed, is also an MLP. Its output is passed through a sigmoid activation and then clamped to a range of $(-10, 10)$ to prevent excessively large score estimates. During the training of this score model using the Denoising Score Identity (DSI), the target score (derived from $\nabla_y \log p_\gamma(y | x)$) is clamped to a maximum norm of 1 (i.e., 'max norm' for 1) to enhance training stability.

4.2.2 Analysis

MOG40 Dataset

On the MOG-40 dataset, a two-step learned Langevin sampler demonstrated good fitting to the target distribution. The first step provides an initial approximation, which is then refined by the second step, leading to improved coverage of the multiple modes, as shown in Figure 4.3.

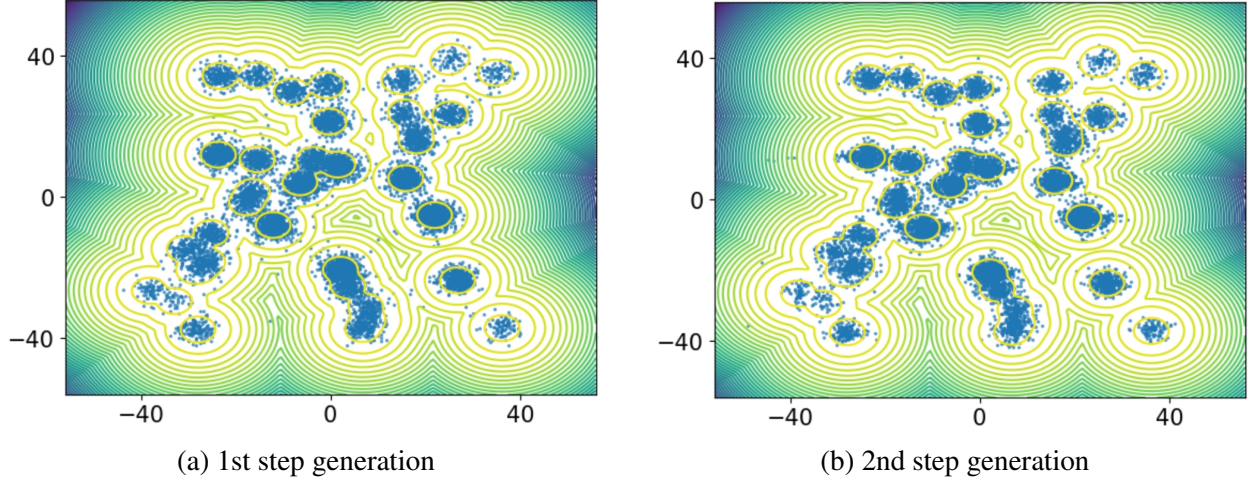


Figure 4.3: Generation results for a 1-step Learned Sampler (LS) and a 2-step LS on the MOG-40 dataset, illustrating improved mode coverage with multiple steps.

The effectiveness of iterative refinement is further substantiated by experiments employing an auto-regressive application of the Langevin sampling framework. As depicted in Figure 4.4, applying a second-stage model in an auto-regressive manner, where initial samples are already partially refined from a preceding stage, consistently leads to improved sample quality over successive iterations. Notably, the enhancements observed per additional step in this auto-regressive setup significantly surpass those achieved with a multi-step DiKL approach, thereby underscoring the superior efficacy of the Langevin sampling framework in this context. Furthermore, the auto-regressive framework demonstrates substantial computational efficiency, with training times on an NVIDIA L4 GPU recorded at approximately 2 hours, representing a five-fold reduction compared to the DiKL method.

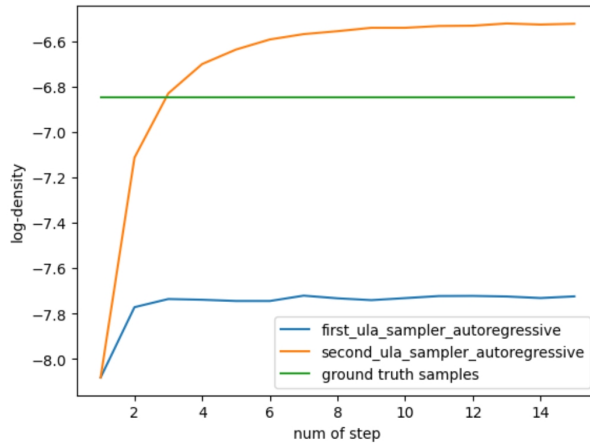


Figure 4.4: Effect of auto-regressive application on the MOG-40 dataset, comparing the performance of repeatedly applying the first-stage sampler versus the second-stage sampler.

However, during auto-regressive sampling on the MOG-40 dataset, a phenomenon of "peak collapse" or mode collapse was observed under certain conditions, as illustrated in Figure 4.5(a). This observation is further substantiated by findings (detailed in Figure 4.8 of an earlier discussion) indicating that the log-density achieved by the neural sampler can artifactually exceed that of the ground truth distribution. Such behavior is attributed to the effect of the inherent bias in the uncorrected Langevin SDE, as discussed in Section 4.1.3.

The introduction of the theoretical correction term, $\nabla_x \gamma(x)$ into the dynamics was found to effectively resolve this peak collapse issue, leading to a more accurate representation of the target distribution, depicted in Figure 4.5(b).

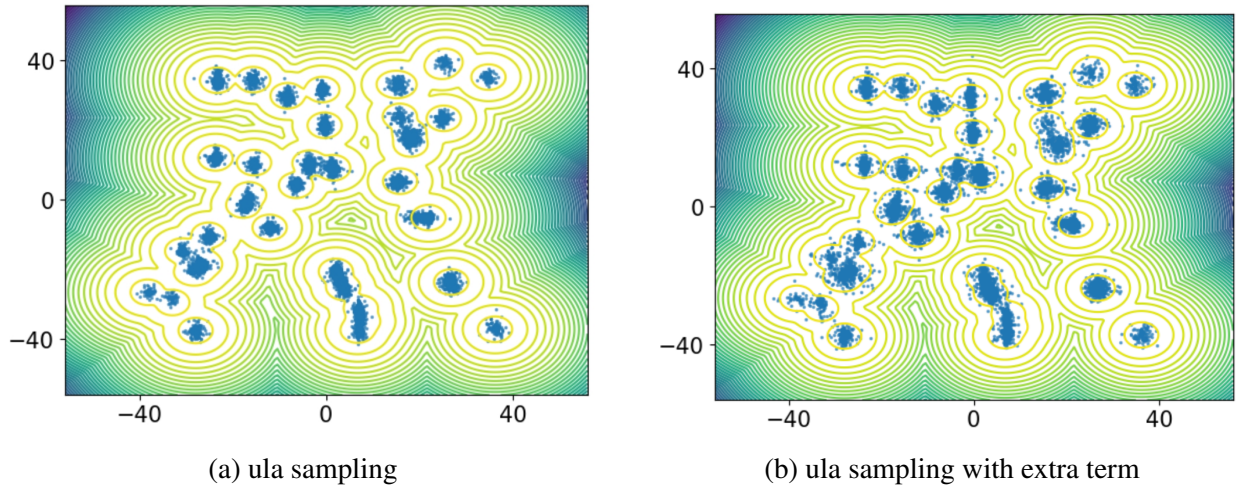


Figure 4.5: Illustration of peak collapse phenomenon during auto-regressive Langevin sampling on the MOG-40 dataset after several iterations and effect of involvement the extra term to solve the bias

MW32 Dataset

For the MW-32 dataset, experiments were conducted primarily in the post-processing regime, where initial samples are noisy versions of target samples. In this setting, it was found that auto-regressive application of the first trained sampler was often sufficient to achieve considerable improvement in sample quality. Figure 4.6 compares the effect of auto-regressively applying the first-stage versus a second-stage sampler.

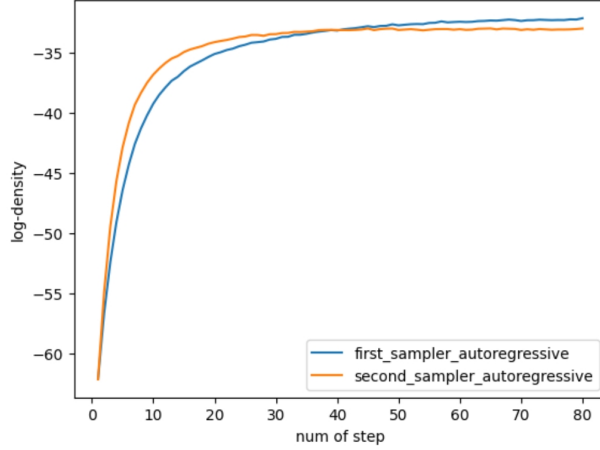


Figure 4.6: Comparison of auto-regressive application of a first-stage versus a second-stage Langevin sampler on the MW-32 dataset (post-processing setup).

When the first-stage Langevin sampler was applied auto-regressively for an extended duration (e.g., 500 steps) on the MW-32 dataset, it yielded high-quality samples, as depicted in Figure 4.7. Notably, the quality of these samples significantly surpassed those obtained using the multi-step DiKL approach discussed in previous chapters.

An interesting observation on the MW-32 dataset is that when the generated samples are in proximity to the ground truth distribution, the learned step-size function $\gamma(x)$ tends to be very smooth. This smoothness implies that the SDE bias correction term $\nabla_x \gamma(x)$ becomes exceedingly small; for instance, its norm was found to be approximately 10^{-5} times the norm of x in such regions. Consequently, the practical effect of this specific bias correction term is minimal. This is further evidenced by the close similarity in sampling quality between the Unadjusted Langevin Algorithm (ULA) results (Figure 4.7) and those obtained when the $\nabla_x \gamma(x)$ correction term is included (Figure 4.8). However, despite the negligible impact of the SDE bias related to a position-dependent $\gamma(x)$ under these conditions, the inherent discretization bias of the ULA scheme persists, preventing a perfect fit to the target distribution.

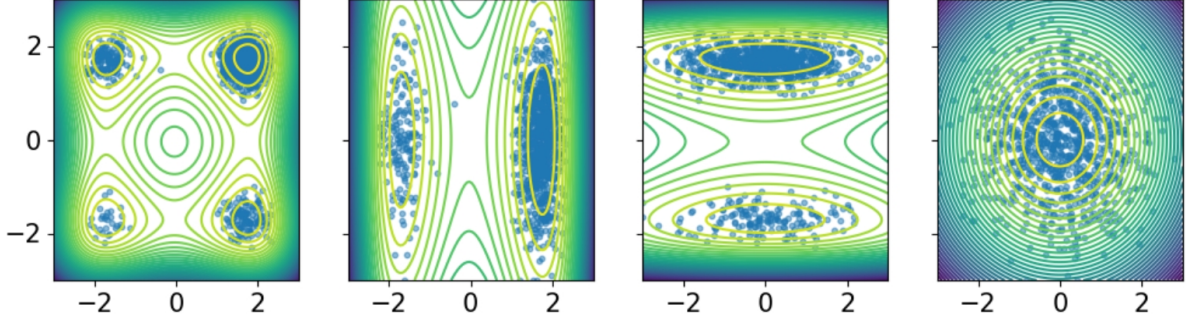


Figure 4.7: Generated samples from the MW-32 dataset after 500 auto-regressive steps using the first-stage Langevin sampler (Unadjusted Langevin Algorithm).

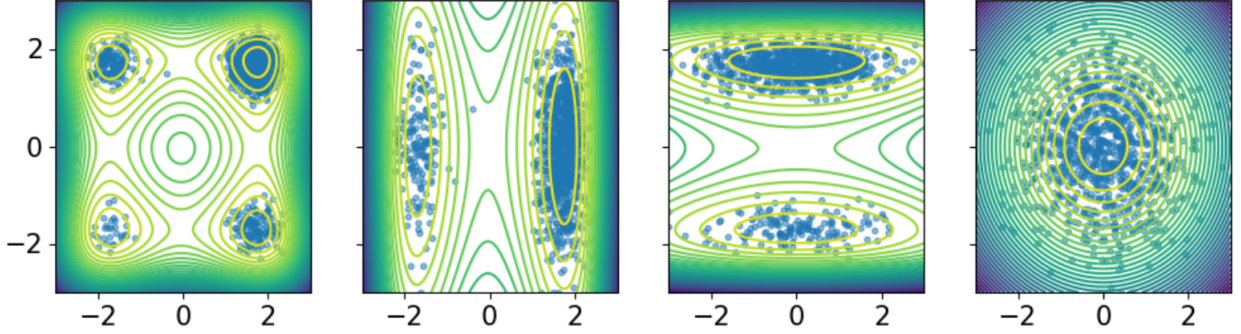


Figure 4.8: Generated samples from the MW-32 dataset after 500 auto-regressive steps using the first-stage Langevin sampler with the SDE bias correction term $\nabla_x \gamma(x)$ included.

4.3 Summary

This chapter explored **Langevin samplers** with learned, position-dependent step sizes ($\gamma(x)$), optimized via KL divergence. To enhance generative capacity, **multi-step and auto-regressive** strategies were detailed. The **bias** inherent in using a position-dependent $\gamma(x)$ within the standard SDE was a key theoretical consideration; while a **correction term** ($\nabla \gamma(x)$) can theoretically ensure convergence, its significant computational cost was noted.

Experimental evaluations on MOG-40 and MW-32 consistently demonstrated that auto-regressive sampling improved sample quality. For MOG-40, this included mitigating peak collapse when the correction term was applied. Critically, across both datasets, these Langevin-based methods provided superior gains in sample quality per step, achieved higher overall sample quality, and realized a five-fold reduction in computational time compared to the multi-step DiKL method.

Chapter 5

Metropolis-Adjusted Langevin Sampler with Learned Dynamics

Introduction

The preceding chapters have examined various neural sampling techniques, including multi-step DiKL and the Unadjusted Langevin Algorithm (ULA) featuring learned, position-dependent step sizes. While ULA leverages gradient information from the target distribution’s log-density, its inherent discretization bias means it generally does not achieve exact convergence, and rectifying this bias can be impractical for complex target systems. This chapter, therefore, shifts focus to the Metropolis-Adjusted Langevin Algorithm (MALA). MALA is a well-established Markov Chain Monte Carlo (MCMC) method that enhances Langevin dynamics to facilitate exact (asymptotic) sampling, notably without an explicit requirement for the target’s energy function. This is accomplished by integrating a Metropolis-Hastings acceptance-rejection step, which systematically corrects for the discretization errors introduced by the Langevin-based proposals.

Building upon the framework of learning adaptive, position-dependent step-size functions $\gamma(\mathbf{x})$ discussed for ULA in Chapter 4, this chapter extends this concept to the MALA framework. We investigate whether equipping MALA with such a learnable $\gamma(\mathbf{x})$ can enhance its efficiency and performance, particularly in navigating complex, high-dimensional probability landscapes. The primary aim is to develop a MALA sampler that not only retains its theoretical guarantee of exact convergence but also benefits from the adaptive exploration capabilities afforded by a neural network-parameterized step-size function.

This chapter will detail the methodology for constructing and optimizing a MALA sampler with a learnable $\gamma(\mathbf{x})$. This includes the formulation of the MALA transition kernel, the Langevin-based proposal mechanism, the Metropolis-Hastings acceptance criteria, and an objective function

based on Kullback-Leibler (KL) divergence for learning $\gamma(\mathbf{x})$. We will also discuss the score function associated with the MALA transition. Following this, the application of multi-step and auto-regressive sampling schemes, analogous to those presented for Langevin samplers in Chapter 4 (Sections 4.1.1 and 4.1.2) will be outlined for the learned MALA framework.

Finally, the chapter presents a comprehensive empirical evaluation of the proposed learned MALA sampler. Using the MOG-40 and MW-32 benchmark datasets, as detailed in the experimental setup (analogous to Chapter 4, Section 4.2.1) we analyze the performance of the sampler, its behavior in multi-step and auto-regressive contexts, and critically compare it against ULA-based approaches to highlight the distinct advantages conferred by the MALA-specific kernel learning and the Metropolis-Hastings correction. Key aspects such as mode coverage, efficiency, and the impact of the Metropolis adjustment on phenomena like peak collapse will be examined.

5.1 Method for Learning Adaptive MALA

The Metropolis-Adjusted Langevin Algorithm (MALA) represents a refinement of Langevin dynamics, integrating gradient-informed proposals with a Metropolis-Hastings acceptance-rejection step. This ensures that the resultant Markov chain converges to the exact target distribution $p_d(\mathbf{x})$, thereby mitigating the inherent bias found in the Unadjusted Langevin Algorithm (ULA) and reducing critical reliance on a perfectly tuned step size, as proposals that would lead to inaccurate sampling are stochastically filtered. This section extends the concept of adaptively learning a position-dependent step size, $\gamma(\mathbf{x})$, previously discussed for Langevin samplers, to the MALA framework.

The MALA transition kernel, $T(\mathbf{y}|\mathbf{x})$, which defines the probability of moving from a state $\mathbf{x}$ to a state $\mathbf{y}$, is characterized by a two-stage process:

$$T(\mathbf{y} | \mathbf{x}) = q(\mathbf{y} | \mathbf{x}) \alpha(\mathbf{x}, \mathbf{y}) + \delta_{\mathbf{x}}(\mathbf{y}) \left(1 - \int q(\mathbf{y}' | \mathbf{x}) \alpha(\mathbf{x}, \mathbf{y}') d\mathbf{y}' \right). \quad (5.1)$$

Here, $q(\mathbf{y} | \mathbf{x})$ is the proposal distribution density for suggesting state $\mathbf{y}$ from state $\mathbf{x}$, $\alpha(\mathbf{x}, \mathbf{y})$ is the Metropolis-Hastings acceptance probability, and $\delta_{\mathbf{x}}(\mathbf{y})$ is the Dirac delta function, which accounts for the probability of the chain remaining at state $\mathbf{x}$ if the proposed move $\mathbf{y}'$ is rejected.

The proposal distribution $q(\mathbf{y} | \mathbf{x})$ is generated using a Langevin-type step, incorporating the learnable, position-dependent step size $\gamma(\mathbf{x})$:

$$q(\mathbf{y} | \mathbf{x}) = p_{\gamma}(\mathbf{y}|\mathbf{x}) = \mathcal{N}(\mathbf{y}; \mathbf{x} + \gamma(\mathbf{x})\nabla \log p_d(\mathbf{x}), 2\gamma(\mathbf{x})\mathbf{I}). \quad (5.2)$$

In this formulation, $\mathbf{x}$ is the current sample, $\nabla \log p_d(\mathbf{x})$ is the score function of the target data distribution $p_d(\mathbf{x})$, and $\mathbf{I}$ is the identity matrix, signifying isotropic noise for the proposal.

The acceptance probability $\alpha(\mathbf{x}, \mathbf{y})$ is determined by the Metropolis-Hastings ratio, $r(\mathbf{x}, \mathbf{y})$:

$$r(\mathbf{x}, \mathbf{y}) = \frac{p_d(\mathbf{y}) q(\mathbf{x} | \mathbf{y})}{p_d(\mathbf{x}) q(\mathbf{y} | \mathbf{x})}, \quad (5.3)$$

where $q(\mathbf{x} | \mathbf{y})$ is the probability density of proposing to move from $\mathbf{y}$ back to $\mathbf{x}$, calculated analogously to Equation 5.2. The acceptance probability is then:

$$\alpha(\mathbf{x}, \mathbf{y}) = \min\{1, r(\mathbf{x}, \mathbf{y})\}. \quad (5.4)$$

This mechanism ensures detailed balance with respect to $p_d(\mathbf{x})$, guaranteeing convergence to the target distribution.

To optimize the learnable step size function $\gamma(\mathbf{x})$, the objective is to minimize the Kullback-Leibler (KL) divergence between the distribution generated by one MALA step, $p_{\gamma|\theta}^m(\mathbf{y})$, and the target data distribution $p_d(\mathbf{y})$. The distribution $p_{\gamma|\theta}^m(\mathbf{y})$ results from applying the MALA transition kernel $T_\gamma(\mathbf{y}|\mathbf{x})$ to an initial distribution of samples $p_\theta(\mathbf{x})$ (e.g., from a base model or a previous sampling stage):

$$p_{\gamma|\theta}^m(\mathbf{y}) = \int T_\gamma(\mathbf{y}|\mathbf{x}) p_\theta(\mathbf{x}) d\mathbf{x}. \quad (5.5)$$

The optimization problem for $\gamma(\mathbf{x})$ is thus formulated as:

$$\gamma^*(\cdot) = \operatorname{argmin}_{\gamma(\cdot)} \mathbb{D}_{\text{KL}}(p_{\gamma|\theta}^m \| p_d). \quad (5.6)$$

Analogous to the optimization strategy employed for the Diffusive KL (DiKL) divergence or direct KL minimization for Langevin samplers, the gradient of this KL divergence with respect to the parameters governing $\gamma(\mathbf{x})$ can be expressed. If we consider $\mathbf{y}$ as the sample resulting from the MALA step, and if its dependence on γ can be appropriately defined for differentiation, the gradient is formally written as:

$$\nabla_\gamma \mathbb{D}_{\text{KL}}(p_{\gamma|\theta}^m \| p_d) = E_{\mathbf{x} \sim p_\theta(\mathbf{x})} E_{\mathbf{y} \sim T_\gamma(\cdot|\mathbf{x})} \left[(\nabla_\mathbf{y} \log p_{\gamma|\theta}^m(\mathbf{y}) - \nabla_\mathbf{y} \log p_d(\mathbf{y})) \frac{\partial \mathbf{y}}{\partial \gamma} \right]. \quad (5.7)$$

The term $\frac{\partial \mathbf{y}}{\partial \gamma}$ is complex due to the stochastic acceptance step in MALA; its practical computation often involves specific approximations or pathwise derivative estimators applied primarily to the proposal stage, or alternative optimization strategies focusing on acceptance rates. A critical component for evaluating this gradient is the score function of the one-step MALA distribution, $\nabla_\mathbf{y} \log p_{\gamma|\theta}^m(\mathbf{y})$.

Analyzing the score of the continuous part of the MALA kernel (i.e., for accepted moves,

$\mathbf{y} \neq \mathbf{x}$), we consider $\nabla_{\mathbf{y}} \log (q(\mathbf{y} | \mathbf{x})\alpha(\mathbf{x}, \mathbf{y}))$. This can be decomposed as:

$$\nabla_{\mathbf{y}} \log (q(\mathbf{y} | \mathbf{x})\alpha(\mathbf{x}, \mathbf{y})) = \nabla_{\mathbf{y}} \log q(\mathbf{y} | \mathbf{x}) + \nabla_{\mathbf{y}} \log \alpha(\mathbf{x}, \mathbf{y}). \quad (5.8)$$

The gradient of the log acceptance probability, $\nabla_{\mathbf{y}} \log \alpha(\mathbf{x}, \mathbf{y})$, exhibits different forms based on $r(\mathbf{x}, \mathbf{y})$:

$$\nabla_{\mathbf{y}} \log \alpha(\mathbf{x}, \mathbf{y}) = \begin{cases} \nabla_{\mathbf{y}} \log r(\mathbf{x}, \mathbf{y}) & \text{if } r(\mathbf{x}, \mathbf{y}) < 1, \\ 0 & \text{if } r(\mathbf{x}, \mathbf{y}) > 1, \\ \text{undefined} & \text{if } r(\mathbf{x}, \mathbf{y}) = 1. \end{cases} \quad (5.9)$$

Substituting $\log r(\mathbf{x}, \mathbf{y}) = \log p_d(\mathbf{y}) + \log q(\mathbf{x} | \mathbf{y}) - \log p_d(\mathbf{x}) - \log q(\mathbf{y} | \mathbf{x})$, the score of an accepted move (continuous part of the kernel) becomes:

$$\nabla_{\mathbf{y}} \log (q(\mathbf{y} | \mathbf{x})\alpha(\mathbf{x}, \mathbf{y})) = \begin{cases} \nabla_{\mathbf{y}} \log p_d(\mathbf{y}) + \nabla_{\mathbf{y}} \log q(\mathbf{x} | \mathbf{y}) & \text{if } r(\mathbf{x}, \mathbf{y}) < 1, \\ \nabla_{\mathbf{y}} \log q(\mathbf{y} | \mathbf{x}) & \text{if } r(\mathbf{x}, \mathbf{y}) > 1, \\ \text{undefined} & \text{if } r(\mathbf{x}, \mathbf{y}) = 1. \end{cases} \quad (5.10)$$

This analysis (where $\nabla_{\mathbf{y}} \log p_d(\mathbf{x}) = 0$ simplifies the expression for $r(\mathbf{x}, \mathbf{y}) < 1$) reveals that when an "uphill" move (in terms of the MALA ratio, meaning $r(\mathbf{x}, \mathbf{y}) < 1$ for a move from x to y) is accepted, the effective score guiding the sample involves the score of the target distribution $p_d(\mathbf{y})$. Estimating terms like $\nabla_{\mathbf{y}} \log p_{\gamma|\theta}^m(\mathbf{y})$ or components thereof often requires techniques analogous to Denoising Score Matching (DSM), as detailed in the background chapter (Section 2.3.1).

5.1.1 Multi-Step and Auto-Regressive MALA Samplers

To further enhance the sampling capabilities of the learned Metropolis-Adjusted Langevin Algorithm (MALA), the concepts of multi-step sampling and auto-regressive application, previously detailed for Langevin samplers (Chapter 4, Sections 1.1 and 1.2) can be directly adopted.

The **multi-step MALA** approach involves serially connecting multiple MALA samplers, each characterized by its own (potentially learned) position-dependent step-size function $\gamma_k(\mathbf{x})$. The samples generated by the k -th MALA sampler serve as the initial input for the $(k + 1)$ -th sampler. During the training or optimization of a particular stage, the parameters of all preceding MALA samplers (i.e., their respective $\gamma_j(\mathbf{x})$ for $j < k$) are held constant. This layered refinement aims to progressively improve the quality of samples, with each MALA step ensuring exactness with respect to its proposal conditioned on the Metropolis-Hastings filter. The conceptual framework for constructing and optimizing such a cascade mirrors that of multi-step Langevin sampling.

Similarly, the **auto-regressive MALA** paradigm involves the iterative application of a single,

trained MALA sampler (with its learned $\gamma(\mathbf{x})$) to its own output. This strategy leverages the learned dynamics encoded within the single MALA model to refine samples over multiple iterations. The primary advantages, such as model parsimony and simplified training compared to training a series of distinct models, are analogous to those observed in auto-regressive Langevin sampling. Each MALA iteration in the auto-regressive chain performs a proposal based on Equation 5.2 and an acceptance step according to Equation 5.4, using the output of the previous iteration as its starting point.

The underlying principles, potential benefits (e.g., increased expressive power, targeted refinement), and implementation logic for both multi-step and auto-regressive MALA samplers are fundamentally identical to those discussed in the context of pure Langevin samplers. Therefore, a detailed re-elaboration of these strategies is omitted here for brevity, and the reader is referred to the comprehensive discussion in the preceding chapter on Langevin Samplers.

5.2 Experiments and Results

5.2.1 Experiment Setting

The empirical evaluation of the learned Metropolis-Adjusted Langevin Algorithm (MALA) sampler, including its multi-step and auto-regressive variants, is conducted using an experimental setup that closely mirrors the one employed for the Langevin sampler evaluations (detailed in Chapter 4, Section 4.2.1). This consistency allows for a comparative understanding of the MALA-based approaches relative to their pure Langevin counterparts.

The specific choices for hyperparameters, network dimensions, training protocols, and evaluation metrics are kept consistent with those used for the Langevin sampler experiments to facilitate direct comparison, unless deviations are explicitly noted for MALA-specific investigations. For a comprehensive description of these common settings, the reader is referred to the detailed experimental setup outlined in the Langevin Sampler chapter.

5.2.2 Analysis of Learned MALA Performance

This section presents an empirical analysis of the learned Metropolis-Adjusted Langevin Algorithm (MALA) sampler, focusing on its performance on the MOG-40 and MW-32 benchmark datasets. We investigate the efficacy of multi-step and auto-regressive strategies and conduct a comparative study against standard Langevin sampling to elucidate the specific benefits of the MALA kernel and the Metropolis-Hastings adjustment.

Performance on MOG-40 Dataset

For the MOG-40 dataset, a two-step learned MALA sampler was trained. The generation results after the first and second steps are illustrated in Figure 5.1, demonstrating the progressive refinement of the sample distribution.

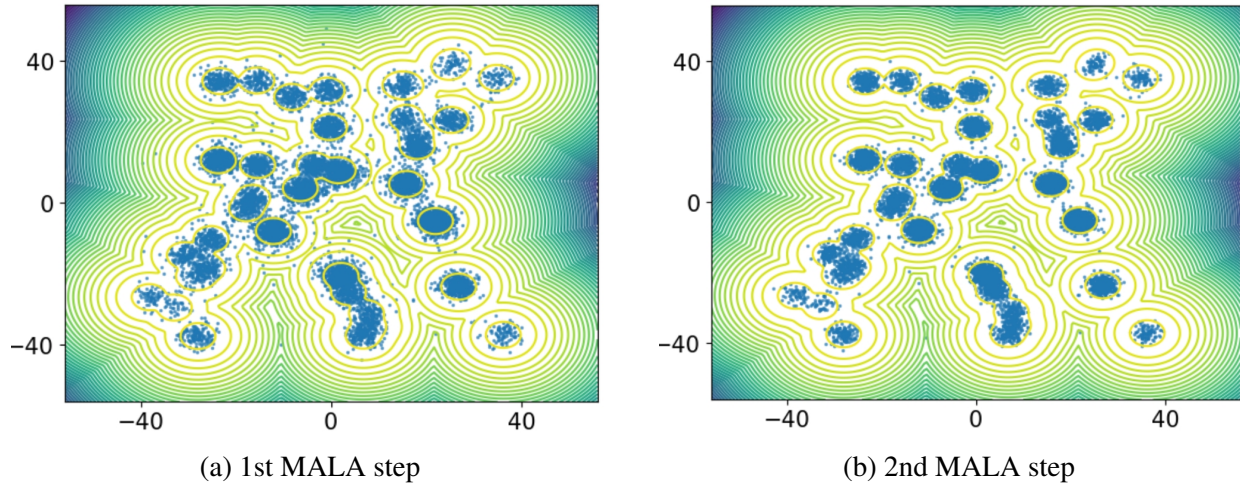


Figure 5.1: Generation results for the learned MALA sampler on the MOG-40 dataset after the 1st and 2nd steps, showing sample refinement and mode coverage.

An important observation is that the model obtained after just the first MALA step often proves sufficiently robust for effective auto-regressive sampling. This implies potential computational savings, as a single well-trained MALA stage can be iteratively applied for further refinement, rather than requiring multiple distinct, sequentially trained models. Figure 5.2 showcases the performance of such auto-regressive application.

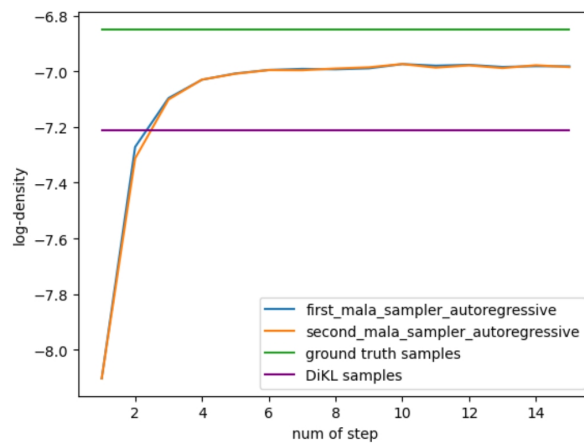


Figure 5.2: Performance of auto-regressive sampling using the first-step MALA model on the MOG-40 dataset. The plot likely illustrates a performance metric (e.g., log density) as a function of auto-regressive steps or in comparison to a multi-step non-auto-regressive approach.

A significant advantage of the Metropolis-Adjusted Langevin Algorithm (MALA) is its inherent ability to mitigate mode collapse. This robustness stems from the Metropolis-Hastings acceptance criterion, which enforces detailed balance and thereby promotes a more comprehensive exploration of the target distribution, as illustrated in Figure 5.3. It is observed that the MALA sampler typically achieves significantly superior sample quality compared to the Unadjusted Langevin Algorithm (ULA) even when the latter incorporates a bias correction term.

Regarding the observed sample distributions, it is noted that samples may appear more concentrated in central peak regions. This phenomenon can be attributed to the characteristics of the initial noise distribution, which, as depicted in Figure 4.1 (from a previous discussion), primarily covers these central areas. A similar tendency for sample concentration influenced by the initial noise was also documented for the standard Langevin sampler, as shown in Figure 4.5.

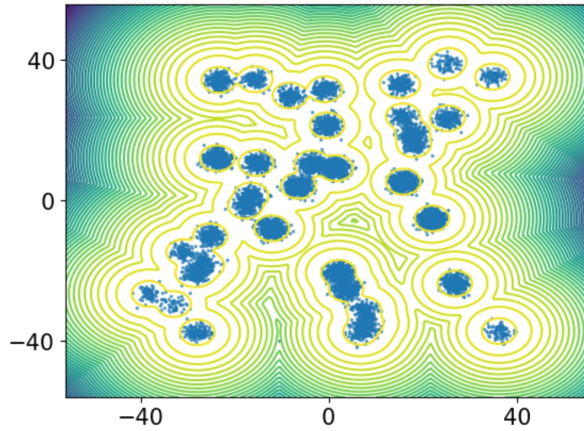


Figure 5.3: Sample distribution after 15 auto-regressive steps using the first-stage MALA model on MOG-40, demonstrating the absence of peak collapse due to the Metropolis adjustment.

Performance on MW-32 Dataset

On the more challenging MW-32 dataset, the learned MALA sampler, particularly when applied auto-regressively in a post-processing context, demonstrated very good fitting to the target distribution. Figure 5.4 displays the marginal distributions after 500 auto-regressive MALA steps. A qualitative assessment of these marginals (e.g., considering the leftmost plot in the figure array) reveals that the sampler accurately captures most modes; for instance, three of the four prominent peaks might be very well approximated, with only minor imperfections in fitting the remaining peak (e.g., the rightmost one). This indicates a strong capability to handle complex, multi-modal landscapes.

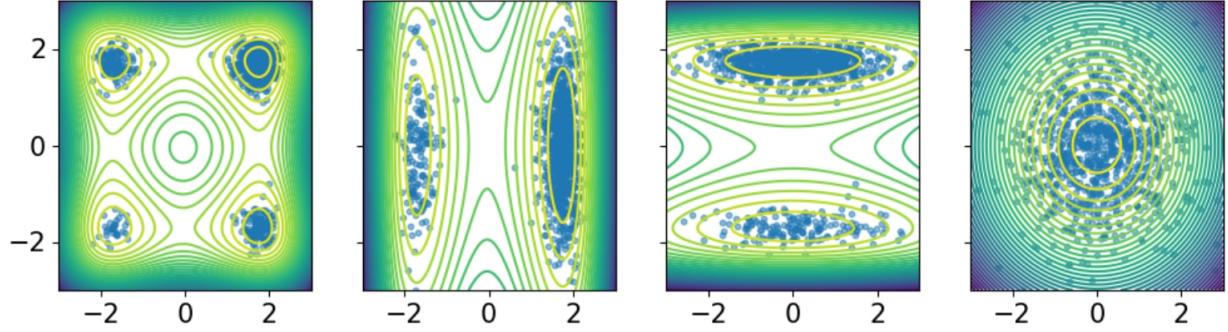


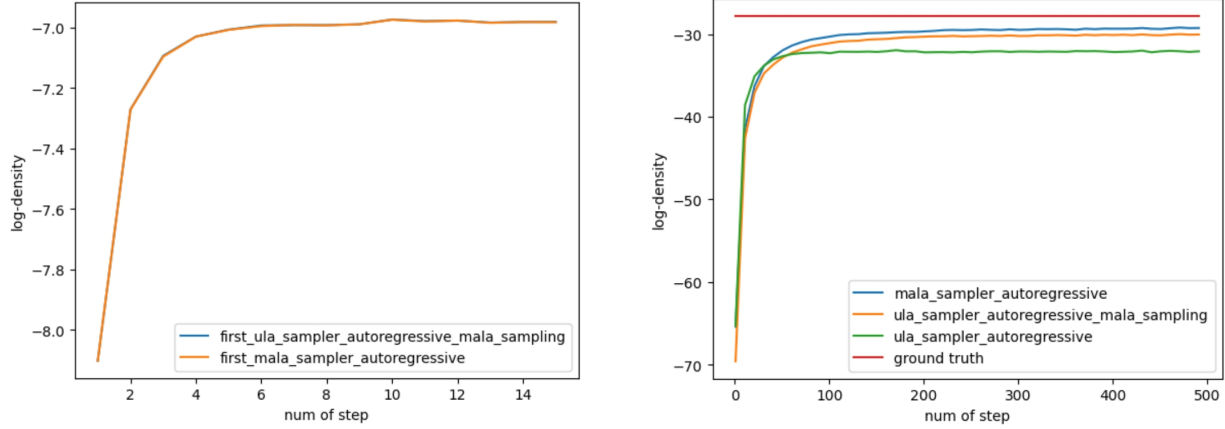
Figure 5.4: Marginal sample distributions from the MW-32 dataset after 500 auto-regressive steps using a learned MALA sampler, illustrating high-fidelity fitting to the complex target.

Comparative Analysis: MALA Kernel versus ULA Kernel with Metropolis-Hastings Adjustment

To discern whether the benefits observed stem from specifically training a MALA kernel (i.e., learning $\gamma(\mathbf{x})$ optimized for the MALA process) or merely from applying the Metropolis-Hastings (MH) correction to any Langevin-type proposal, a comparative study was conducted. Three distinct approaches were evaluated:

1. **MALA Kernel with MALA Sampling (Learned MALA):** The step-size function $\gamma(\mathbf{x})$ is learned with an objective suited for MALA (method in this chapter), and sampling proceeds with MH adjustment.
2. **ULA Kernel with MALA Sampling (ULA Proposal + MH):** The step-size function $\gamma(\mathbf{x})$ is learned as if for a standard Unadjusted Langevin Algorithm (ULA) (the method in last chapter), but an MH acceptance step is applied post-hoc to the ULA proposals.
3. **ULA Kernel with ULA Sampling (Learned ULA):** Standard learned ULA without any MH adjustment.

The results of these comparisons, presented in Figure 5.5, vary based on the complexity of the target distribution.



(a) Performance on MOG-40 (log density vs. steps). (b) Performance on MW-32 (log density vs. steps).

Figure 5.5: Comparative performance (e.g., log density over auto-regressive steps) of different sampling strategies: MALA Kernel + MALA Sampling (blue), ULA Kernel + MALA Sampling (orange), and ULA Kernel + ULA Sampling (green) on the MOG-40 and MW-32 datasets.

On the relatively simpler MOG-40 dataset (Figure 5.5a), the performance of the learned MALA kernel is nearly indistinguishable from that of using a ULA-trained kernel with an MH adjustment. This suggests that when the sampling task is sufficiently easy, the acceptance rates for ULA proposals are already high, as the ULA sampler can effectively reach regions of high probability. As discussed in Section 5.1 (Equation 5.10), if most proposals are accepted (i.e., $r(\mathbf{x}, \mathbf{y}) > 1$), the score guiding the MALA update potentially leads to a learning objective for $\gamma(\mathbf{x})$ that behaves similarly to that for a ULA kernel, especially if optimization focuses on accepted proposals. Consequently, a MALA-specific kernel offers little additional benefit over a ULA kernel when the MH filter is universally applied and mostly accepts.

Conversely, on the more challenging MW-32 dataset (Figure 5.5b), the advantages of a specifically trained MALA kernel become evident. The ‘MALA Kernel + MALA Sampling’ strategy (represented by the blue line in typical comparative plots) consistently demonstrates the best performance, outperforming both ‘ULA Kernel + MALA Sampling’ (orange line) and ‘ULA Kernel + ULA Sampling’ (green line). This improved performance is attributed to the MALA kernel’s $\gamma(\mathbf{x})$ being optimized not just for proposal generation but implicitly or explicitly for maximizing or maintaining a high MH acceptance ratio, leading to more efficient exploration of the state space while preserving exactness.

Interestingly, for a small number of auto-regressive steps (e.g., fewer than 50, as potentially depicted by the orange line dipping below the green line in Figure 5.5b), applying MALA sampling to a ULA-trained kernel can initially perform worse than straightforward ULA sampling. This is because the ULA kernel is not optimized to produce proposals that are frequently accepted by an MH filter tailored for $p_d(\mathbf{x})$. Consequently, a higher rejection rate can stymie progress compared

to ULA, which always accepts its (biased) proposals. However, as the number of steps increases (e.g., beyond 50), the samples generally move closer to regions of higher target probability. In this regime, the task locally simplifies, and the acceptance rate for the ULA proposals under an MH filter improves. Here, the benefit of the MH correction (ensuring exactness) allows the ‘ULA Kernel + MALA Sampling’ (orange line) to surpass the biased ‘ULA Kernel + ULA Sampling’ (green line). Nevertheless, the ‘MALA Kernel + MALA Sampling’ approach (blue line) consistently outperforms the ‘ULA Kernel + MALA Sampling’ (orange line), underscoring the value of learning a proposal mechanism ($\gamma(x)$) that is co-adapted with the Metropolis-Hastings acceptance criteria.

5.3 Summary

This chapter investigated the Metropolis-Adjusted Langevin Algorithm (MALA) augmented with a learnable, position-dependent step-size function $\gamma(x)$, aiming to merge MALA’s exactness with the efficiencies of learned adaptive dynamics. The core methodology involved Langevin-based proposals using the learned $\gamma(x)$, a Metropolis-Hastings acceptance step ensuring convergence to the target $p_d(x)$, and a KL divergence-based objective for optimizing $\gamma(x)$. Multi-step and auto-regressive sampling paradigms, analogous to those for Langevin samplers, were noted as applicable for enhanced performance.

Empirical evaluations on MOG-40 and MW-32 datasets demonstrated that the learned MALA sampler performed effectively. On MOG-40, auto-regressive sampling initiated from the first MALA step proved robust and notably prevented mode collapse due to the Metropolis-Hastings adjustment. For the complex MW-32 dataset, auto-regressive MALA achieved high-fidelity fitting, particularly as a post-processor.

A key insight from comparative analyses was the significant benefit of training a MALA-specific kernel (i.e., learning $\gamma(x)$ specifically for MALA) on challenging datasets like MW-32. This MALA-specific kernel, by co-adapting the proposal mechanism with the acceptance criteria, substantially outperformed applying a Metropolis-Hastings step to proposals from a ULA-trained kernel or standard ULA. While this specific kernel optimization was less critical for simpler tasks where ULA proposals had high acceptance rates, its advantage on complex distributions was clear.

In conclusion, learned MALA with an adaptive step-size function offers a powerful approach for exact sampling. It successfully integrates MALA’s theoretical guarantees with neural network-driven flexibility. The findings highlight that while the Metropolis-Hastings filter provides inherent robustness, co-optimizing the proposal mechanism through a learned MALA kernel is crucial for achieving superior performance on challenging, high-dimensional target distributions, positioning learned MALA as a valuable advanced sampling tool.

Chapter 6

Conclusion

This thesis addressed the critical challenge of efficient and accurate sampling from complex, high-dimensional Boltzmann distributions, particularly relevant to molecular modeling. We proposed and evaluated advanced neural sampler architectures aimed at improving generative model performance.

Initially, we explored a **multi-step Diffusive KL (DiKL) sampler**, demonstrating that iterative refinement improved sample quality, albeit with diminishing returns and linear computational costs.

Building on this, we introduced a novel **Langevin sampler with learned dynamics**, which incorporates a position-dependent step size and for which we mathematically analyzed its bias. This methodology substantially improved sample quality and computational efficiency, yielding superior performance gains per step compared to the multi-step DiKL. Its auto-regressive application further reduced training times, although an occasional tendency towards mode-seeking behavior was observed.

Finally, we developed a **Metropolis-Adjusted Langevin sampler with learned dynamics**. This method combined adaptive Langevin proposals with Metropolis-Hastings acceptance, ensuring rigorous convergence guarantees while maintaining efficiency. Experimental results on challenging datasets confirmed the superior performance of these Langevin-based methods in both sample quality and computational speed.

In summary, this thesis developed and validated novel Langevin-based neural samplers with learned dynamics. These contributions offer a promising direction for enhancing generative modeling, providing more accurate and efficient sampling techniques crucial for complex scientific applications.

6.1 Limitations

A notable limitation of the proposed multi-step methodologies, particularly evident in the sequentially trained models, is the inherent inter-dependency between training stages. Specifically, the

training of a model for a subsequent step cannot commence until the optimization of the preceding model is fully completed. While the auto-regressive framework partially mitigates this issue by allowing the repeated application of a single, already trained model, this dependency can still pose challenges in more demanding scenarios, potentially hindering parallelization and overall training efficiency for highly complex or multi-stage refinement tasks.

6.2 Future Works

We outline several additional directions for future investigation:

- **Extension to Other Langevin-Based Methods:** Beyond the Unadjusted Langevin Algorithm (ULA) and Metropolis-Adjusted Langevin Algorithm (MALA) explored in this thesis, the principle of incorporating learnable, position-dependent step sizes can be extended to a broader family of Langevin-based sampling methods. This includes, but is not limited to, stochastic gradient Langevin dynamics (SGLD), Hamiltonian Monte Carlo (HMC), or even variants with adaptive noise structures. Investigating the efficacy and theoretical properties of learned dynamics within these diverse frameworks represents a rich area for future research.
- **Analytical Convergence Speed Analysis:** A rigorous theoretical analysis of the convergence speed for this family of neural samplers with learnable step sizes is crucial. Assuming optimal training at each step, an analytical study of how the convergence rate scales with the number of steps and the complexity of the target distribution would provide valuable insights into the fundamental limits and efficiency gains of these methods. This theoretical understanding would complement empirical observations and guide future algorithm design.
- **Leveraging Necessary Conditions for Parallel Training:** The inherent structure of Langevin dynamics, particularly the necessary conditions for optimal step size when each step is perfectly optimized (i.e., when the gradient of the step size with respect to the optimization objective is zero), provides valuable information. This theoretical insight could potentially be leveraged to formulate novel loss functions or training paradigms. By incorporating these optimality conditions, it might be possible to develop alternative training objectives that facilitate parallel training for this class of neural samplers, thereby directly addressing the current sequential dependency limitation. This could involve designing loss terms that encourage individual steps to converge towards their locally optimal step sizes, even when trained in parallel, ultimately contributing to a more scalable and efficient training process.

Bibliography

- [1] David H Ackley, Geoffrey E Hinton, and Terrence J Sejnowski. A learning algorithm for boltzmann machines. *Cognitive science*, 9(1):147–169, 1985.
- [2] Minkyung Baek, Frank DiMaio, Ivan Anishchenko, Justas Dauparas, Sergey Ovchinnikov, Gyu Rie Lee, Jue Wang, Qian Cong, Lisa N Kinch, R Dustin Schaeffer, et al. Accurate prediction of protein structures and interactions using a three-track neural network. *Science*, 373(6557):871–876, 2021.
- [3] David Baker and Andrej Sali. Protein structure prediction and structural genomics. *Science*, 294(5540):93–96, 2001.
- [4] Rafael C Bernardi, Marcelo CR Melo, and Klaus Schulten. Enhanced sampling techniques in molecular dynamics simulations of biological systems. *Biochimica et Biophysica Acta (BBA)-General Subjects*, 1850(5):872–877, 2015.
- [5] Julius Berner, Lorenz Richter, and Karen Ullrich. An optimal control perspective on diffusion-based generative modeling. *arXiv preprint arXiv:2211.01364*, 2022.
- [6] Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*, volume 4. Springer, 2006.
- [7] Wenlin Chen, Mingtian Zhang, Brooks Paige, José Miguel Hernández-Lobato, and David Barber. Diffusive gibbs sampling. *arXiv preprint arXiv:2402.03008*, 2024.
- [8] Stefan Chmiela, Huziel E Sauceda, Klaus-Robert Müller, and Alexandre Tkatchenko. Towards exact molecular dynamics simulations with machine-learned force fields. *Nature communications*, 9(1):3887, 2018.
- [9] Lauro Langosco di Langosco, Vincent Fortuin, and Heiko Strathmann. Neural variational gradient descent. *arXiv preprint arXiv:2107.10731*, 2021.
- [10] Miguel A González. Force fields and molecular dynamics simulations. *École thématique de la Société Française de la Neutronique*, 12:169–200, 2011.

- [11] Jiajun He, Wenlin Chen, Mingtian Zhang, David Barber, and José Miguel Hernández-Lobato. Training neural samplers with reverse diffusive kl divergence. *arXiv preprint arXiv:2410.12456*, 2024.
- [12] Jiajun He, Yuanqi Du, Francisco Vargas, Dinghuai Zhang, Shreyas Padhy, RuiKang OuYang, Carla Gomes, and José Miguel Hernández-Lobato. No trick, no treat: Pursuits and challenges towards simulation-free training of neural samplers. *arXiv preprint arXiv:2502.06685*, 2025.
- [13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.
- [14] James P Hughes, Stephen Rees, S Barrett Kalindjian, and Karen L Philpott. Principles of early drug discovery. *British journal of pharmacology*, 162(6):1239–1249, 2011.
- [15] Ferenc Huszár. Variational inference using implicit distributions. *arXiv preprint arXiv:1702.08235*, 2017.
- [16] Leon Klein, Andrew Foong, Tor Fjelde, Bruno Mlodozieniec, Marc Brockschmidt, Sebastian Nowozin, Frank Noé, and Ryota Tomioka. Timewarp: Transferable acceleration of molecular dynamics by learning time-coarsened dynamics. *Advances in Neural Information Processing Systems*, 36:52863–52883, 2023.
- [17] Leon Klein and Frank Noé. Transferable boltzmann generators. *arXiv preprint arXiv:2406.14426*, 2024.
- [18] Laurence Midgley, Vincent Stimper, Javier Antorán, Emile Mathieu, Bernhard Schölkopf, and José Miguel Hernández-Lobato. Se (3) equivariant augmented coupling flows. *Advances in Neural Information Processing Systems*, 36:79200–79225, 2023.
- [19] Laurence Illing Midgley, Vincent Stimper, Gregor NC Simm, Bernhard Schölkopf, and José Miguel Hernández-Lobato. Flow annealed importance sampling bootstrap. *arXiv preprint arXiv:2208.01893*, 2022.
- [20] Radford M Neal et al. Mcmc using hamiltonian dynamics. *Handbook of markov chain monte carlo*, 2(11):2, 2011.
- [21] Frank Noé, Simon Olsson, Jonas Köhler, and Hao Wu. Boltzmann generators: Sampling equilibrium states of many-body systems with deep learning. *Science*, 365(6457):eaaw1147, 2019.

- [22] Sebastian Nowozin, Botond Cseke, and Ryota Tomioka. f-gan: Training generative neural samplers using variational divergence minimization. *Advances in neural information processing systems*, 29, 2016.
- [23] Jean-Francois Richard and Wei Zhang. Efficient high-dimensional importance sampling. *Journal of Econometrics*, 141(2):1385–1411, 2007.
- [24] Tamar Schlick. *Molecular modeling and simulation: an interdisciplinary guide*, volume 2. Springer, 2010.
- [25] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.
- [26] Jiří Šponer, Giovanni Bussi, Miroslav Krepl, Pavel Banáš, Sandro Bottaro, Richard A Cunha, Alejandro Gil-Ley, Giovanni Pinamonti, Simón Poblete, Petr Jurečka, et al. Rna structural dynamics as captured by molecular simulations: a comprehensive overview. *Chemical Reviews*, 118(8):4177, 2018.
- [27] Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural computation*, 23(7):1661–1674, 2011.
- [28] Gregory A Voth. *Coarse-graining of condensed phase and biomolecular systems*. CRC press, 2008.
- [29] Gert Vriend. What if: a molecular modeling and drug design program. *Journal of molecular graphics*, 8(1):52–56, 1990.
- [30] Jejoong Yoo, David Winogradoff, and Aleksei Aksimentiev. Molecular dynamics simulations of dna–dna and dna–protein interactions. *Current Opinion in Structural Biology*, 64:88–96, 2020.
- [31] Mingtian Zhang, Peter Hayes, Thomas Bird, Raza Habib, and David Barber. Spread divergence. In *International Conference on Machine Learning*, pages 11106–11116. PMLR, 2020.

Appendix A

Risk Assessment

The initial 4th Year Project Hazard Assessment Form submitted at the start of Michaelmas term indicated that no specific hazards were anticipated beyond general laboratory/office environment considerations. The project, focusing on computational methods for molecular sampling (Gaussian convolution loss and Langevin sampling), primarily involved computer use.

Alignment with Encountered Hazards

The primary hazard encountered, inherent to a computational project, was related to prolonged Display Screen Equipment (DSE) use. This includes potential for musculoskeletal discomfort (e.g., from posture) and visual fatigue. While specific project-related hazards beyond the general work environment were not initially detailed, the nature of the project inherently involved significant computer use, a common consideration. In practice, managing screen time, ensuring appropriate workstation ergonomics, and taking regular breaks were necessary to mitigate these DSE-related risks, which are common in such research. No other significant, unlisted hazards were encountered.

Reflections on the Risk Assessment Process

If undertaking a similar project again, the risk assessment should be the same, indicating no specific project-related hazards beyond common DSE considerations.